

DE VOS Victor

DOSSIER DE PROJET

FORMATION DWWM 2024-2025

Stage au sein de l'entreprise Baïfall Dream



Sommaire

Sommaire.....	2
À mon propos.....	4
Contexte.....	5
Le projet Allo Afro Food.....	5
Les besoins.....	5
Les contraintes.....	6
Les méthodes de travail.....	6
La mise en oeuvre.....	7
Cahier des charges.....	7
Objectifs du site.....	7
Fonctionnalités principales.....	8
Supports visés.....	8
Arborescence du site.....	9
Back-office.....	9
Spécifications techniques du projet.....	10
Applicatif.....	10
Langages & Frameworks.....	10
Symfony et ses dépendances.....	12
API et bibliothèques JavaScript externes.....	12
Réalisation du projet.....	13
Les tâches front-end.....	13
L'installation et la configuration du projet.....	13
Création des maquettes des pages du site.....	16
Réalisation de nos interfaces utilisateurs statiques	

web.....	17
Réalisation des parties dynamiques de nos pages...	22
Les tâches back-end.....	25
Conception & évolution de la base de données.....	25
Automatisation et gestion des données avec Symfony.....	26
Développement des fonctionnalités serveur et gestion des données.....	27
Documentation et gestion du code source.....	29
Sécurité et maintenance du site.....	30
Mesures mises en place contre les principales failles de sécurité.....	31
Application des tests fonctionnels et unitaires.....	32
Tests fonctionnels.....	32
Tests unitaires.....	33
En conclusion.....	34
ANNEXES.....	35

À mon propos

Avant de présenter mon projet, je souhaite me présenter et expliquer les choix qui m'ont conduit à me former en Développement Web et Web Mobile.

Issu d'un parcours en Lettres Classiques et Histoire, j'ai longtemps envisagé mon avenir dans le domaine littéraire. Pourtant, au fil du temps, l'analyse de textes et l'exégèse d'œuvres ont fini par me sembler trop abstraites. J'ai alors ressenti le besoin de me tourner vers un domaine plus concret, où je pourrais allier réflexion et application pratique.

C'est ainsi que j'ai découvert la rédaction technique, un métier mêlant écriture, langage et ingénierie. Lors d'une alternance dans une entreprise spécialisée en cybersécurité, j'ai plongé dans l'univers du digital et de l'informatique. Les premiers mois furent exigeants, mais j'ai rapidement pris goût à cet environnement stimulant.

Après une seconde expérience en entreprise et une veille approfondie sur mon métier, j'ai constaté que la maîtrise du code était un atout précieux pour un rédacteur technique. Un professionnel capable de comprendre, écrire et optimiser du code apporte une véritable valeur ajoutée à la documentation technique. Cette réflexion m'a conduit à me former au développement web en intégrant l'AFPA Territoire Digital en juin 2024.

Aujourd'hui, mon objectif est de combiner mes compétences en rédaction et en développement.

Dans ce dossier, je vous présente le travail réalisé durant ma formation et plus précisément lors de mon stage au sein de l'entreprise **Baïfall Dream**.

Le Contexte

Le projet Allo Afro Food

Baïfall Dream est fondée en 1992 par l'artiste pluridisciplinaire franco-sénégalais **Mike Sylla**. À la base, Baïfall Dream c'est avant tout un concept qui fédère un collectif d'artistes autour de la mode. Puis, au fil du temps, l'entreprise s'est étendue autour des activités de Mike Sylla telles que la couture, l'évènementiel ou encore la radio.

Depuis quelques années, Mike Sylla a pour ambition d'ouvrir un restaurant/traiteur et une épicerie africaine à Paris. Son objectif est de promouvoir et de rendre la nourriture africaine accessible à tous. Il travaille notamment avec des producteurs sénégalais pour les produits importés de la future épicerie. Ce projet a pour nom **Allo Afro Food**.

Après quelques échanges téléphoniques avec ma tutrice de stage, **Mme. Gorenzsch**, qui est notamment la coordinatrice projets pour Baïfall Dream, j'ai eu l'opportunité de proposer à l'une de mes camarades de formation **Mariem Tlich** de venir réaliser son stage dans la compagnie. Nous avons ainsi pu collaborer tous les deux au développement du projet de janvier à mars 2025.

Les contraintes

L'une des principales contraintes était que **le restaurant Allo Afro Food n'existe pas encore** et certains partenaires et collaborateurs d'Allo Afro Food, tels que l'équipe cuisine et les prestataires de livraison, étaient encore en discussion. Cela signifiait que **de nombreuses informations restaient à définir**, nous obligeant à **générer du contenu fictif** (images, menus, produits de l'épicerie, services traiteur, informations de contact) et à avancer parfois un petit peu à l'aveugle sur certains sujets (mode de livraison, possibilité de commander des produits du restaurant et de l'épicerie ensemble).

Une autre contrainte importante était que Mme. Gorenzsch ne voulait pas faire appel à une entreprise tierce pour automatiser des tâches comme la gestion des commandes, le stock des produits et autres car elle souhaitait avoir le plus de contrôle possible sur le code. Nous avons essayé de comparer certaines API avec Mariem pour répondre à ces besoins mais ne nous ne sommes pas allés au bout de ce travail par manque de temps mais aussi par manque de compétences techniques.

Les méthodes de travail

Notre tutrice nous a laissé une **grande liberté d'organisation**. Nous faisons un **point hebdomadaire** avec elle pour discuter des avancées, des problématiques rencontrées et des solutions possibles.

Au début du stage, **Mariem et moi communiquons quotidiennement** afin de poser les bases du projet :

- Définition des objectifs et questions en amont.
- Installation de l'environnement de développement.
- Élaboration de la **base de données**.
- Mise en place de **branches Git** pour chaque fonctionnalité.
- Gestion des éventuels problèmes techniques liés à la configuration.

Une fois ces fondations établies, nous avons pu nous **répartir les tâches** et organiser notre travail via **Trello**, un outil de gestion de projet collaboratif.

La mise en oeuvre

Dès le début du projet, **Mariam et moi** avons convenu de créer un **nouvel environnement** de développement, et ce pour plusieurs raisons :

1. Une **première version d'Allo Afro Food** avait été créée par un apprenti développeur il y a quelques années. **Le code existant étant en PHP procédural**, il était préférable de mettre en application nos compétences en **Symfony** pour garantir une meilleure maintenabilité et évolutivité du projet.
2. Nous souhaitions avoir **un accès total à notre environnement** pour éviter les contraintes techniques imposées par la plateforme d'hébergement VirtualMin utilisée par l'ancien développeur.
3. **Baïfall Dream** possédant un compte **Hostinger**, nous avons jugé pertinent de créer une **base de données directement hébergée sur cette plateforme**, facilitant ainsi la gestion et l'évolution du projet.

Le cahier des charges

Après une discussion avec **Mme Gorenszach**, nous avons défini un **cahier des charges** à partir des besoins exprimés.

Objectifs du site

Développer un site web permettant aux utilisateurs de :

- Commander **des plats et des produits de l'épicerie**.
- Envoyer **des devis personnalisés** pour les services traiteur.
- **Créer et gérer leur compte** sur le site.

Fonctionnalités principales

- Gestion du panier et de la wish list pour les items du restaurant et de l'épicerie.
- Gestion des devis personnalisés pour la partie traiteur.
- Gestion du compte utilisateur : commandes, devis, messages, profil.
- Gestion du système de paiement des commandes.
- Gestion du back-office pour l'administrateur.

Quelques exemples de sites que nous avons visités :

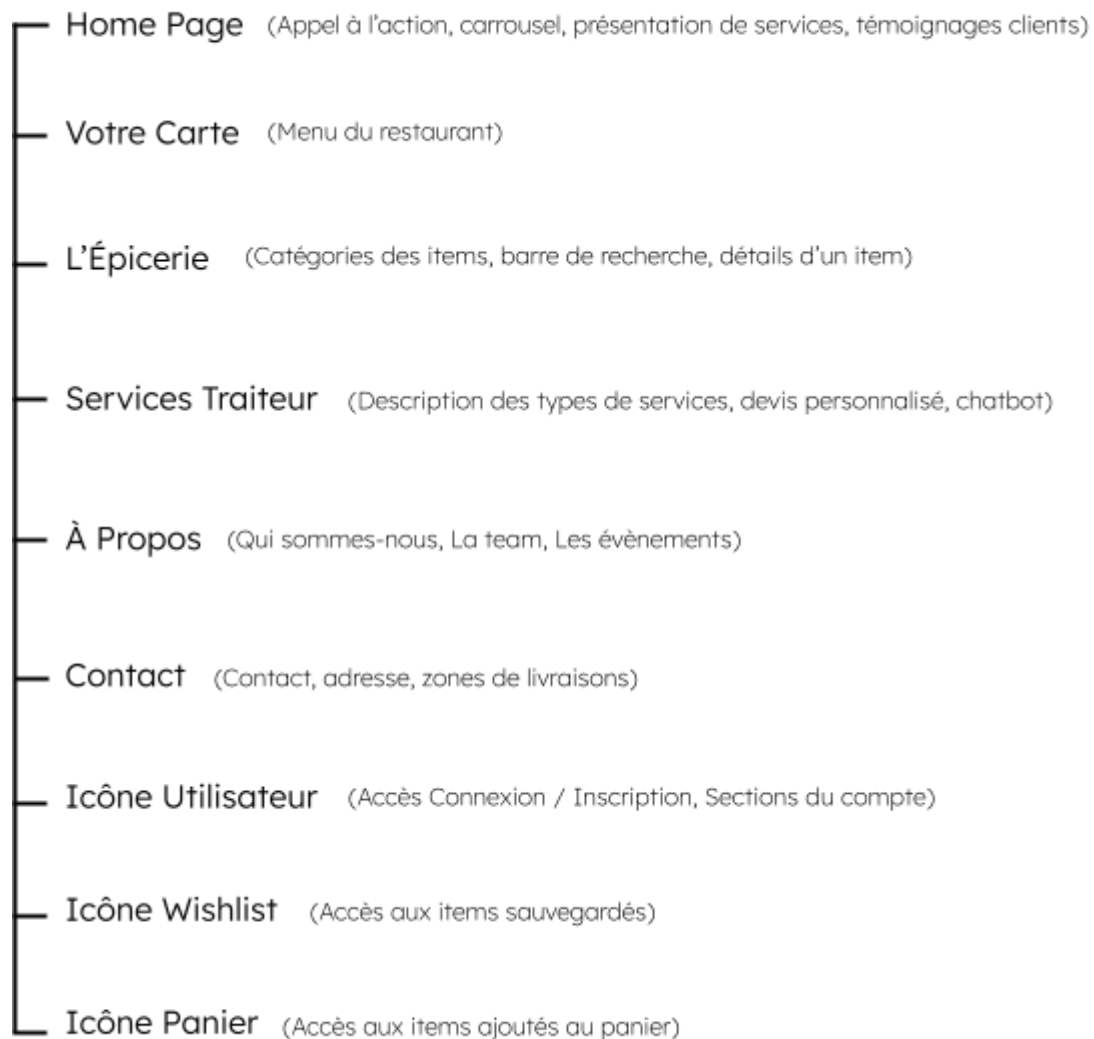
- <https://maison-sore.com/>
- <https://www.afriknfusion.fr/>
- <https://www.bmkparis.com/>

Supports visés

- Ordinateurs de bureau (Desktop)
- Ordinateurs portables

Nous nous sommes principalement concentrés sur ces supports pour le stage. Une application mobile *Allo Afro Food* sera en cours de développement dans un futur proche et nous n'avons pas perfectionné la responsivité du site, bien que la plus grande partie le soit déjà avec les classes Bootstrap. En effet, le temps du stage étant relativement court, notre tutrice a souhaité se concentrer sur la version desktop du site.

Arborescence du site



Back-office

- Accessible par les utilisateurs ayant les droits administrateur.
- Pouvoir ajouter, consulter, modifier ou supprimer les catégories des 3 services principaux du site et leurs items respectifs.
- Pouvoir consulter les utilisateurs du site.
- Pouvoir consulter ou modifier les commandes des utilisateurs.

Spécifications techniques du projet

Outils applicatifs



IDE de prédilection pour Mariem et moi-même, ses extensions sont très pratiques pour formater les différents langages et pour nous indiquer les éventuelles erreurs.



Dans l'optique de créer nos maquettes, l'outil de collaboration **Figma** s'est imposé naturellement. Aussi, pendant le stage, un étudiant graphiste, Jude, a pu facilement se greffer au projet en collaborant via l'outil.



Nous avons utilisé **GitHub** pour le développement du projet. Comme nous étions 2 à travailler dessus, il fallait pouvoir se répartir les tâches, créer plusieurs branches et versionner notre code. GitHub possède une interface utilisateur moderne et intuitive et nous connaissions déjà l'outil.

Langages & Frameworks



Les 2 langages composent la base du site, **HTML5** permettant de structurer l'information et **CSS3** permettant de mettre en forme et de styliser nos pages.



Nous avons utilisé **SASS**, un langage de script préprocesseur nous permettant d'intégrer des variables et des mixins dans notre code et par-dessus tout de compiler l'intégralité de notre CSS en un seul fichier via Webpack Encore, un outil basé sur NPM permettant de compiler les fichiers lors de la mise en production.

Bootstrap 5

En plus de nos feuilles de style personnalisées, nous avons décidé d'importer le framework front-end **Bootstrap** qui nous a permis d'homogénéiser nos pages via ses classes, de faciliter le responsive et d'utiliser son extension JavaScript pour certaines de ses fonctionnalités natives (modales, offcanvas, toasts).

JavaScript

Nous avons utilisé **JavaScript** pour dynamiser nos pages et pour fluidifier l'expérience utilisateur. Nous avons également compilé tous les fichiers .js avec Webpack Encore.

PHP 8

Nous avons choisi d'utiliser **PHP** car nous avons besoin de produire des pages dynamiques, pour traiter les formulaires, pour accéder facilement à la base de données et que nous voulions un langage orienté objet pour mettre en pratique ce que nous avons appris en formation.

SQL

Nous utilisons du **SQL** pour dialoguer avec la base de données, notamment pour exécuter des requêtes manuelles directement sur phpMyAdmin ou lors de l'édition de fichiers de migrations sur Symfony.

Symfony et ses dépendances



Nous avons donc opté pour **Symfony** pour le projet car c'est un framework écrit en PHP objet, séparant le code en 3 couches de par le **modèle MVC** (Model, Vue, Controller). Il contient des outils apportant des fonctionnalités comme un **ORM** (via Doctrine) et un moteur de template (via **Twig**). Il utilise aussi des bundles pour gérer de très nombreuses fonctionnalités intéressantes pour notre projet.

Twig

Twig est un moteur de template intégré par défaut dans Symfony pour créer des pages HTML recevant du PHP. Il permet de réaliser des templates plus propres, car il n'y a plus aucune balise PHP dans les pages HTML mais seulement des variables entre `{{ }}`. On peut également faire hériter un template sur un autre grâce à son système de blocs.

Doctrine

Doctrine est l'ORM (Object-Relational Mapping) intégré par défaut dans Symfony. Cela nous a permis de gérer nos données.

Doctrine s'est chargée de créer la base, les tables, les relations et de gérer les données en fonction des classes créées.

API et bibliothèques JavaScript externes

Stripe

Pour la gestion du paiement, nous avons intégré l'API de **Stripe**. L'API est fiable, open-source, plutôt simple d'utilisation et sa documentation est très bien faite.

Leaflet, intl-tel-input, AOS

Nous avons intégré les bibliothèques JavaScript suivantes :

- **Leaflet** pour la localisation du restaurant sur une carte interactive.
- **intl-tel-input** pour afficher les indicatifs téléphoniques dans les formulaires selon les pays.
- **AOS** pour un réaliser un effet de transition entre deux sections sur une même page.

Réalisation du projet

En amont de la réalisation du projet, nous avons créé un nom de domaine pour le site (alloafrofood.com) et une base de données via l'hébergeur **Hostinger**.

A la fin du stage, pour partager notre travail avec l'équipe mais aussi pour les futures dev sur le projet, nous avons créé un sous-répertoire *alloafrofood/test* dans lequel nous avons transféré notre développement (depuis une branche *test*).

Le front-end

L'installation et la configuration du projet

Mes missions pour installer et configurer le projet ont été de :

- Créer un repository GitHub pour le développement
- Cloner le repository avec le dossier du projet Symfony
- Créer le projet Symfony en local

Créer un repository GitHub

Comme nous partions d'un tout nouveau développement, nous avons besoin d'un repository GitHub autre que celui de la première version développée par l'ancien étudiant. Nous avons donc mis en place un repository avec notre tutrice.

Le compte GitHub étant celui de l'entreprise Baïfall Dream, Mme. Gorenszach nous a octroyé les droits nécessaires pour éditer le répertoire.

Cloner le repository avec le dossier du projet

Afin de stocker le projet sur GitHub, il fallait configurer mon dossier en local pour l'instant vide, avec l'adresse du repository. Pour se faire, j'ai dû cloner le repository dans le chemin du projet. On ouvre donc un terminal GIT en veillant à se situer dans le bon chemin et on exécute la commande suivante :

```
Victor@WIN-0JP1CAATNT6 MINGW64 ~/Desktop/AAF/REPO  
o $ git clone https://github.com/baifalldream/AlloafrofoodV2.git
```

Créer le projet Symfony en local

Comme nous avons déjà vu Symfony durant la formation, nous avons tous les prérequis pour son installation, à savoir Composer, Git et une version PHP supérieure ou égale à 8.1. Même s'il existait des versions supérieures, nous avons initié le projet avec Symfony en version 6.4 car c'est la dernière version **LTS** (Long Term Supported) du framework, autrement dit une version bénéficiant d'un support étendu, avec des mises à jour de sécurité et des corrections de bugs critiques sur une durée plus longue que les versions standard. Nous avons également choisi d'utiliser la version en **CLI** (Command Line Interface) car nous trouvons plus simple le fait d'exécuter des commandes.

Il ne restait qu'à créer le projet Symfony avec la commande suivante :

```
symfony new AlloAfroFoodV2 --version=lts --webapp
```

- En CLI, on écrit **symfony** au lieu de **php bin/console**.
- **AlloAfroFoodV2** est le titre du projet.
- **-version=lts** signifie que nous installons le projet en version LTS.

- **-webapp** installe Symfony avec tous les packages de bases et des packages additionnels.

Notre projet Symfony étant créé, nous retrouvons les dossiers et fichiers habituels d'un projet Symfony, avec les éléments principaux et caractéristiques de Symfony :

- Le dossier **src** comportant les dossiers Controller, Entity et Repository.
- Le dossier **templates** comportant les fichiers Twig.
- Le fichier **composer.json** comportant toutes les dépendances PHP de mon projet.
- Le fichier **package.json** comportant toutes les dépendances JavaScript de mon projet.

Voir en annexe :

- [Annexe 1 : Racine du projet \(p.38\)](#)

Pour s'assurer que le projet Symfony est bien connecté à mon serveur local, il suffit d'exécuter la commande suivante dans le chemin du projet :

```
PS C:\Users\Victor\Desktop\AAF\REPO\AlloAfroFoodV2> symfony server:start
```

Si j'installe une dépendance en local, elle sera ajoutée dans le dossier **node_modules**, qui est présent par défaut dans le fichier **.gitignore** (contenu ignoré par Git). Ainsi, lorsque ma collègue récupérera mon travail (et vice versa) via la commande **git pull**, elle devra exécuter la commande **npm install** pour installer les dépendances manquantes.

Création des maquettes des pages du site

Recueil de la demande

La partie "maquettes" du projet s'est opérée en trois temps. Dans un premier temps, nous avons recueilli les demandes de M. Sylla et Mme Gorenszach concernant les couleurs principales du site, ainsi que les typographies et pictogrammes à utiliser. À partir de ces retours, nous

avons proposé différentes couleurs, teintes, typographies et pictogrammes, que nous avons réunis sur **Figma**.

Voir en annexe :

- [Annexe 2 : Proposition de design \(p.39\)](#)

Réalisation des premières maquettes

Dans un second temps, Mariem et moi avons réalisé les premières maquettes, en collaboration étroite avec notre tutrice, pour affiner nos choix et nous assurer qu'ils répondaient aux attentes du projet. Bien que ces maquettes ne soient pas particulièrement esthétiques, elles ont été un outil précieux pour clarifier l'ordonnancement des éléments et définir les fonctionnalités à intégrer sur les pages principales du site. Ces échanges ont permis de poser les bases du design avant de commencer le développement des pages.

Voir en annexe :

- [Annexe 3 : Premières maquettes \(p.40\)](#)

Arrivé de Jude, étudiant web Designer

Enfin, dans un troisième temps, un étudiant Web Designer, Jude, a rejoint le projet. À ce moment-là, nous avions déjà bien avancé dans le développement. Jude a donc dû faire ses propositions en partant du visuel du site déjà en développement. Nous avons échangé avec lui sur nos idées de développement, ce qui lui a permis de formaliser des maquettes plus esthétiques qui ont été des sources de propositions et d'améliorations pour affiner et perfectionner le design du site.

Voir en annexe :

- [Annexe 4 : Maquettes réalisées avec Jude \(p.41\)](#)

Réalisation de nos interfaces utilisateurs statiques web

L'avantage du framework Symfony est non seulement qu'il est un outil robuste pour la partie back-end, mais qu'il permet aussi une grande flexibilité et une modularité dans la conception de ses pages web.

À l'instar du fichier **base.html.twig** qui a pour objectifs de :

- Définir la structure HTML commune (head, body, scripts...).
- Faciliter l'héritage Twig (définit une relation parent-enfant) pour les autres templates avec `{% extends 'base.html.twig' %}`.
- Inclure des blocs dynamiques pour permettre à ses pages enfants d'ajouter du contenu.

Autrement dit, au lieu d'ajouter manuellement des éléments communs à chacune de nos pages comme des fichiers de scripts, du CSS, un header et un footer, nous n'avons qu'à inclure ces éléments dans notre fichier **base.html.twig** puis à étendre le fichier dans nos pages avec `{% extends 'base.html.twig' %}`.

Voir en annexe :

- [Annexe 5 : Snippet fichier base.html.twig \(p.42\)](#)

Vous remarquerez la condition suivante pour l'inclusion du header et du footer :

```
{% if no_header_footer is not defined or not no_header_footer %}
```

Ici, on indique que si la variable `no_header_footer` n'est pas définie ou si elle est définie mais vaut `false`, alors le header et le footer seront affichés.

Cette condition permet d'éviter une erreur si la variable n'existe pas et d'autoriser nos pages liées au tunnel de vente à masquer le header et le footer avec : `{% set no_header_footer = true %}`.

Il y a 2 commandes très pratiques en Symfony (`symfony console make:entity NomDeLentite`, `symfony console make:controller NomDuController`), l'une consiste à créer une entité et un repository et l'autre consiste à créer un controller et un fichier Twig. Ces 2 commandes sont complémentaires car elles couvrent 2 aspects fondamentaux du développement avec Symfony :

- `make:entity` : Le modèle (données et base de données)

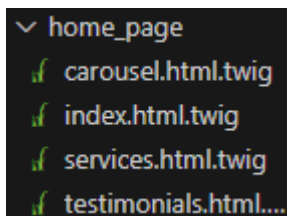
- ◆ Crée une **entité** (classe qui représente une table en base de données).
- ◆ Génère un **repository** pour interagir avec la base de données.
- ◆ Définit les **champs** et leurs types (ex : **nom**, **prix**, **description** pour une entité **Produit**).

→ **make:controller** : La logique (Routes & affichage)

Réalisation de la Homepage et de ses éléments

À partir de la maquette, nous nous sommes attelés à créer la **Homepage**, car elle est centrale et représentative du site web. Nous avons convenu d'y ajouter :

- Une image occupant 100% de la hauteur avec par-dessus un titre et un bouton d'appel à l'action.
- Une section plats du jour avec un carrousel interactif.
- Une section montrant les services principaux du site.
- Une partie témoignage client.



Grâce à la modularité de Symfony précédemment évoquée, nous avons pu découper notre Homepage selon son contenu au lieu d'avoir l'ensemble du code dans un seul et unique fichier.

Ensuite, il ne nous restait plus qu'à inclure nos sections dans le fichier Twig **index.html.twig** regroupant tout le contenu de la page :

```
<div class="container-fluid homepage__carousel_container p-0">
|   {% include 'home_page/carousel.html.twig' with { products: products } %}
</div>
<div class="container-fluid mb-5 p-0">
|   {% include 'home_page/services.html.twig' %}
</div>
<div class="container-fluid homepage__testimonials_container mb-5">
|   {% include 'home_page/testimonials.html.twig' %}
</div>
```

Vous retrouvez ci-dessus la même syntaxe Twig que pour le fichier **base.html.twig** {% %} sauf que dans ce contexte, il n'est pas question

d'**extends** mais d'**include** qui insère un fichier dans un autre sans prendre en compte l'héritage (sans aucune relation hiérarchique).

Voir en annexe :

- [Annexe 6 : Homepage du site \(p.43\)](#)

Gestion du style des nos pages web

Très rapidement s'est posée la question du style et du CSS pour notre projet. Au début, nous avons commencé par créer des fichiers **.css** pour chacune de nos pages / composants en plus d'avoir importé **Bootstrap** et donc d'avoir des styles déjà définis. En faisant des recherches, nous avons pris connaissance de Webpack Encore, un outil qui permet de gérer les assets (CSS, JS, images, fonts, etc.) et notamment de :

- Compiler tous les fichiers SCSS/SASS en un seul fichier CSS.
- Optimiser et *minifier* le JavaScript et le CSS en production.
- Gérer les fichiers statiques (images, fonts).

Dans l'optique de voir un jour le site déployé sur le web, nous avons décidé d'utiliser Webpack Encore pour que le site soit plus performant. Cette décision impliquait plusieurs choses importantes, à savoir de :

- Convertir tous nos fichiers CSS en SCSS.
- Modifier le nom de nos classes et de nos id sur toutes nos pages pour éviter les conflits de style lors de la compilation.

Cas d'usage : Une div **container-fluid** est une div Bootstrap qui contient du CSS par défaut. Nous utilisons cette div sur nos pages mais nous lui adressons **en plus** des propriétés personnalisées. Webpack Encore a pour effet de compiler tout le SCSS en un seul fichier CSS et donc, en l'état des choses, sans ajouter ou renommer nos classes, nous pouvions nous retrouver avec plusieurs **.container-fluid** dans notre CSS global et avoir de nombreux problèmes de style en sortie après la compilation.

Pour remédier à ce problème, il fallait donc ajouter un autre nom de classe pour chaque div `container-fluid`, de telle sorte qu'en plus d'hériter du style par défaut de Bootstrap, on puisse les différencier dans le CSS global :

```
<div class="container-fluid homepage__container">
```

Pour le nommage, nous avons choisi d'utiliser le nom de page puis 2 fois underscore suivi soit du nom de la section de la page (le cas échéant) ou soit du rôle du contenant, ce qui permet de toujours rendre une classe spécifique :

```
.homepage__carousel_container {  
  height: 550px;  
  background-color: #rgb(255, 255, 255);  
}  
.homepage__testimonials_container {  
  height: 400px;  
}
```

En choisissant d'utiliser Webpack Encore, cela nous a permis d'adopter la syntaxe SCSS dont le nesting, les mixins et les variables et ainsi d'avoir un code encore plus flexible et modulaire :

- Une fois les styles imbriqués les uns dans les autres (le nesting), plus de conflits en sortie au niveau des noms. Nous avons pu laisser un simple `a` imbriqué sachant qu'en sortie, il ne causerait pas de conflits avec les autres ancres.
- Nous avons créé un fichier de mixins pour éviter de dupliquer du code. Par exemple, nous utilisons Flexbox dans plusieurs fichiers et notamment `display: flex, justify-content: center, align-items:center`. La mixin nous a permis de ne pas dupliquer à chaque fois ces propriétés.
- Nous avons créé un fichier de variables pour pouvoir les réutiliser à travers toutes nos pages de style sans pour autant les utiliser, car les couleurs, tailles des titres et autres éléments n'ont pas été validés et le code peut encore évoluer.

Enfin, nous avons inclus nos fichiers dans un unique fichier SCSS `appStyle.scss` compilé avec Webpack dans `webpack.config.js`. Ce dernier crée un fichier CSS global que nous importons dans notre fichier `base.html.twig` dont toutes nos pages web héritent.

Voir en annexe :

- [Annexe 7 : Snippet appStyle.scss \(p.44\)](#)
- [Annexe 8 : Snippet webpack.config.js \(p.45\)](#)
- [Annexe 9 : Snippet du block stylesheets \(p.45\)](#)

Création des comptes Utilisateur et Admin

Pour le projet, nous avons besoin de créer deux types d'accès : un accès classique pour les utilisateurs et un accès réservé aux administrateurs. Pour répondre à cette exigence, nous avons utilisé plusieurs fonctionnalités de Symfony qui permettent de gérer les utilisateurs et leurs rôles de manière sécurisée et flexible.

Nous avons commencé par utiliser la commande `make:user` de Symfony, qui permet de créer une entité **User** pré-configurée avec des fonctionnalités natives pour la gestion des rôles et des mots de passe. Cette commande crée également un contrôleur avec des actions de base, telles que la gestion de la connexion (**login**) et de la déconnexion (**logout**). Grâce à cette commande, nous avons pu implémenter facilement un système d'authentification et de gestion des rôles pour distinguer les utilisateurs standards des administrateurs.

Pour gérer l'accès et sécuriser les routes, nous avons configuré le fichier **security.yaml**, un fichier de configuration de sécurité de Symfony. Ce fichier définit les règles de sécurité de l'application, telles que l'accès autorisé ou restreint à certaines routes en fonction des rôles des utilisateurs. Par exemple, nous avons spécifié que seules les utilisateurs ayant l'attribut **ROLE_ADMIN** pouvaient accéder à l'interface d'administration.

Voir en annexe :

- [Annexe 10 : Snippet du fichier security.yaml \(p.46\)](#)

En parallèle, nous avons créé un fichier **LoginAuthenticator.php** pour personnaliser le processus d'authentification. Ce fichier gère l'authentification des utilisateurs via un formulaire de connexion, en vérifiant les identifiants de l'utilisateur et en redirigeant vers la page appropriée en fonction de son rôle. Il étend la classe **AbstractLoginFormAuthenticator** de Symfony, permettant de personnaliser la logique d'authentification selon les besoins spécifiques de notre projet.

Voir en annexe :

- [Annexe 11 : Snippet du fichier LoginAuthenticator \(p.47\)](#)

Nous avons également découvert EasyAdmin, le back-office natif de Symfony qui facilite l'interaction avec la base de données et le front-end pour la gestion des utilisateurs, des commandes et autres entités du projet. Nous avons choisi de l'implémenter pour simplifier la gestion des utilisateurs et l'administration des données dans notre application.

Voir en annexe :

- [Annexe 12 : Interface du back-office EasyAdmin \(p.48\)](#)

En résumé, nous avons utilisé **Symfony Security** pour sécuriser notre application, configuré les rôles dans **security.yaml**, et créé **LoginAuthenticator.php** pour personnaliser l'authentification. Grâce à ces outils, nous avons mis en place une gestion des utilisateurs efficace et sécurisée, tout en utilisant EasyAdmin pour permettre aux administrateurs d'accéder facilement à l'interface d'administration.

Réalisation des parties dynamiques de nos pages

Durant la réalisation du projet, nous voulions essayer de rendre l'expérience utilisateur fluide et intuitive. Un site web ne doit pas seulement afficher des pages, il doit réagir, interagir et simplifier la vie des utilisateurs. Pour illustrer cet aspect dynamique, j'ai choisi de mettre en avant trois exemples concrets qui montrent comment la méthode **AJAX** (Asynchronous JavaScript And XML) et la mise en application d'une **API** (Application Programming Interface) permettent d'apporter cette interactivité.

Connexion utilisateur en AJAX

L'utilisation d'AJAX pour la connexion permet de rendre l'expérience utilisateur beaucoup plus fluide, car elle évite un rechargement complet de la page. Lorsque l'utilisateur soumet ses identifiants, une requête AJAX est envoyée en arrière-plan pour vérifier la connexion. Si l'authentification réussit, une réponse est envoyée au navigateur avec un message de bienvenue. Le script gère alors l'affichage de la nouvelle page, tout en stockant les informations nécessaires dans le navigateur pour éviter une nouvelle saisie.

En cas d'échec, au lieu d'un simple message d'erreur générique, les champs concernés (email et mot de passe) sont mis à jour directement avec des messages d'erreur spécifiques, offrant ainsi une réponse immédiate.

Voir en annexe :

- [Annexe 13 : Screenshot des messages de connexion \(p.49\)](#)

Implémentation de l'API Stripe

L'intégration de Stripe dans notre projet repose sur les principes d'une API REST, qui permet la communication entre différentes applications via des requêtes HTTP. Ici, le serveur Symfony joue le rôle d'intermédiaire entre le client (le navigateur) et le service de paiement Stripe. Lorsque l'utilisateur valide son paiement, le front-end envoie une requête au back-end sous forme de données structurées en JSON. Le serveur traite cette requête, interagit avec Stripe pour sécuriser la transaction, puis renvoie une réponse indiquant si le paiement a réussi ou non. Ce modèle RESTful permet une séparation claire des responsabilités : le client se charge de l'interface et des interactions utilisateur, tandis que le serveur gère la logique métier et la communication avec Stripe.

Voir en annexe :

- [Annexe 14 : Snippets application de Stripe \(p.50\)](#)

Création d'un chatbot

Sur la page Traiteur de notre site, nous avons mis en place un chatbot pour répondre aux questions des utilisateurs concernant les services traiteurs. Nous avons essayé de le concevoir pour comprendre et répondre de manière intelligente aux demandes grâce à plusieurs techniques de traitement du langage naturel.

Les principales capacités du chatbot sont les suivantes :

1. **Comparaison avec un fichier JSON** : Le chatbot récupère des informations d'un fichier JSON contenant des mots-clés et leurs réponses associées. Chaque entrée du JSON contient un ensemble de mots-clés (ex. : "personnes", "invités") et une réponse correspondante (ex. : "Nous acceptons des groupes de 20 à 150 personnes...").
2. **Normalisation du message** : Lorsqu'un utilisateur envoie un message, celui-ci est normalisé, c'est-à-dire converti en minuscules et débarrassé de la ponctuation, afin de faciliter la comparaison avec les mots-clés du JSON. Cette étape garantit que les différences de casse ou de

ponctuation n'affectent pas la compréhension du message.

3. **Gestion des synonymes** : Le chatbot prend en compte les synonymes afin d'élargir la reconnaissance des termes. Par exemple, des mots comme "réserver", "commander" ou "passer commande" sont considérés comme équivalents et renvoient à la même réponse, ce qui permet de mieux comprendre les questions de l'utilisateur.
4. **Suppression des stopwords** : Pour éviter que des mots inutiles comme "le", "la", "et" n'affectent la pertinence du message, le chatbot élimine ces "stopwords" avant de procéder à l'analyse, ce qui améliore la qualité des réponses.
5. **Matching avec Levenshtein** : Une fois les mots-clés extraits et normalisés, le chatbot utilise l'algorithme de distance de Levenshtein pour mesurer la similarité entre les mots du message et ceux présents dans le fichier JSON. Cet algorithme permet de déterminer si un mot est proche d'un mot-clé (même s'il y a des erreurs d'orthographe ou des variations légères) et ainsi d'attribuer une réponse appropriée.
6. **Calcul du score de correspondance** : Le chatbot attribue un "score" à chaque réponse potentielle, en fonction de la similarité des mots et des synonymes. Les mots-clés les plus importants, comme "commander" ou "réservation", ont un poids plus élevé. Ainsi, le chatbot peut renvoyer la réponse la plus pertinente selon les mots utilisés par l'utilisateur.

Le chatbot, bien que fonctionnel, nécessite encore des améliorations pour mieux répondre à une plus large gamme de demandes. Cependant, nous avons pu mettre en place une base solide, qui pourra être affinée et étendue au fil du temps.

Voir en annexe :

- [Annexe 15 : Visuels du chatbot \(p.51-52\)](#)

Le back-end

Conception & évolution de la base de données

Pour construire notre base de données, on a commencé par **modéliser nos entités avec Looping**, un logiciel open-source qui nous a permis de poser une première version du **Modèle Conceptuel de Données (MCD)**. C'était une étape essentielle pour bien comprendre les relations entre les différentes entités avant d'aller plus loin. Une fois cette première structure en place, on l'a traduite en **Modèle Logique de Données (MLD)**, plus proche de ce qui allait être réellement implémenté.

Voir en annexe :

- [Annexe 16 : Représentation de la base de données \(p.53\)](#)

Bien sûr, la base de données n'est pas restée figée. Elle a évolué au fur et à mesure qu'on avançait dans le projet et qu'on ajustait nos besoins. Dès qu'on a eu une première version de notre vue fonctionnelle, on a créé la base sur **Hostinger**, via le compte **Baïfall Dream**. **Hostinger propose MySQL pour gérer les bases de données**, ce qui nous a été très utile puisqu'on était déjà familiers avec **SQL** et **phpMyAdmin**, l'interface qu'Hostinger met à disposition pour gérer les bases.

Ensuite, nous avons récupéré l'**URL et les identifiants** de la base de données Hostinger pour la connecter à notre projet Symfony via le fichier **.env**, où l'on a renseigné ces accès.

Voir en annexe :

- [Annexe 17 : Snippet du fichier .env \(p.53\)](#)

Automatisation et gestion des données avec Symfony

Une fois la base configurée dans le projet, **Symfony facilite grandement la gestion des tables**. Lorsqu'on crée une entité en ligne de commande, le **framework génère automatiquement la structure correspondante**, et en lançant une **migration**, les tables sont directement ajoutées à la base de données sans qu'on ait à les créer manuellement.

Création des entités

Dans Symfony, les entités représentent les tables de la base de données sous forme de classes PHP. Pour en créer une, on utilise la commande suivante dans le terminal :

```
symfony console make:entity
```

Cette commande lance un assistant interactif qui permet de nommer l'entité et de lui ajouter des attributs (champs de la table) avec leur type (string, integer, datetime, etc.). Une fois validée, Symfony génère automatiquement la classe correspondante dans le dossier `src/Entity/`, avec les propriétés et leurs getters/setters.

Génération et exécution des migrations

Après avoir créé ou modifié une entité, il faut appliquer ces changements à la base de données. Symfony utilise Doctrine pour gérer cette étape via un système de migration. Une migration est un fichier qui contient les instructions SQL nécessaires pour synchroniser la base avec le code.

On génère un fichier de migration avec la commande :

```
symfony console make:migration
```

Symfony analyse les différences entre les entités et la base de données, puis crée un fichier dans le dossier `migrations/` avec les instructions SQL adaptées. Chaque fichier de migration possède deux fonctions essentielles :

- `up()` : Cette fonction définit les modifications à appliquer à la base de données, comme la création ou la modification de tables et de colonnes.
- `down()` : Cette fonction permet d'annuler les modifications faites par `up()`, ce qui est utile en cas d'erreur ou si l'on souhaite revenir en arrière.

Voir en annexe :

- [Annexe 18 : Snippet d'un fichier de migration \(p.54\)](#)

Pour appliquer ces modifications, il suffit d'exécuter :

```
symfony console doctrine:migrations:migrate
```

Cette commande applique les migrations et met à jour la base en conséquence.

Développement des fonctionnalités serveur et gestion des données

Système de recherche et création de filtres

Dans le cadre du développement de la marketplace épicerie, il a été nécessaire d'intégrer une **barre de recherche** ainsi qu'un **système de filtres** permettant aux utilisateurs de trouver facilement les produits correspondant à leurs critères. Cette fonctionnalité repose sur une combinaison entre le **controller**, qui récupère les filtres saisis par l'utilisateur, et le **repository**, qui construit la requête dynamique pour interroger la base de données.

Le controller Symfony gère la réception des paramètres de recherche envoyés par l'utilisateur et les transmet au repository.

Voir en annexe :

- [Annexe 19 : Snippet du controller de recherche & filtres \(p.54\)](#)

Dans ce code, la fonction **findByFilters()** du repository est appelée avec les paramètres de recherche. C'est cette méthode qui va réellement construire la requête SQL pour récupérer les produits correspondants.

La méthode **findByFilters()** utilise un **QueryBuilder** pour générer une requête dynamique. Cela permet d'ajouter des filtres uniquement si les paramètres sont renseignés, optimisant ainsi l'exécution de la requête.

Voir en annexe :

- [Annexe 20 : Snippet du repository de recherche & filtres \(p.55\)](#)

Mise en place d'une pagination

Afin d'améliorer l'**expérience utilisateur** et les **performances de la page**, un système de pagination a été mis en place pour limiter le nombre de produits

affichés simultanément sur la marketplace. Cela évite que l'utilisateur **scrolle indéfiniment**, tout en **réduisant la charge sur le serveur** en évitant de charger tous les produits d'un coup.

Pour cela, un **bundle Symfony** a été intégré, et l'interface **PaginatorInterface** est utilisée pour gérer la pagination des produits et limiter le nombre de produits affichés.

voir annexe pagination

paginate() prend en paramètre :

- La requête (QueryBuilder).
- La page actuelle (**page** récupérée dans la requête GET).
- Le nombre d'éléments par page (fixé ici à **8**).
- La variable **pagination** est transmise au template.
- Celui-ci affichera les produits **pagés**, avec des boutons pour naviguer entre les pages.

Voir en annexe :

- [Annexe 21 : Screenshot de la pagination des produits \(p.55\)](#)

Affichage des produits par catégorie

Dans cette partie, l'objectif est d'afficher les **produits du restaurant** classés par catégorie sur la page **Menu**. Pour cela, le **controller** fait appel au **repository**, qui exécute une requête pour récupérer les produits correspondant à chaque catégorie.

Dans le **repository**, la méthode **findByCategory()** permet de récupérer les produits d'une catégorie spécifique en utilisant un **QueryBuilder**.

Voir en annexe :

- [Annexe 22 : Snippet du code du Menu repository \(p.56\)](#)

Détails de la requête :

1. **Création d'une requête** avec **QueryBuilder** sur l'entité **ProductsRestaurant**.

2. Ajout d'une condition **WHERE** pour filtrer par catégorie (`p.category = :category`).
3. Passage du paramètre `category` via `setParameter()`.
4. Exécution de la requête et récupération des résultats sous forme de tableau.

Le controller `index()` gère l'affichage de la page **Menu**.

Voir en annexe :

- [Annexe 23 : Snippet du code du Menu controller \(p.56\)](#)
- La route `/menu` est définie pour afficher la page du menu.
- Le contrôleur appelle `findByCategory()` du `repository` pour récupérer les produits de chaque catégorie.
- Ces résultats sont envoyés au template `menu_restaurant/index.html.twig` pour être affichés.

Voir en annexe :

- [Annexe 24 : Screenshot de la page Menu \(p.57\)](#)

Documentation et gestion du code source

Nous avons mis en place une **documentation complète** afin de faciliter la compréhension, l'évolution et le déploiement de l'application.

Rédaction d'un README détaillé

Un **README principal** a été rédigé à la racine du projet pour :

- Expliquer l'installation et le déploiement de l'application, en tenant compte des **dépendances** et des **versions** des différents composants utilisés (Symfony, Webpack Encore, OpenCart, Stripe, etc.).
- Documenter l'architecture du projet ainsi que les fonctionnalités clés.
- Fournir les **commandes utiles** pour la mise en place de l'environnement de développement et de production.

Voir en annexe :

- [Annexe 25 : Screenshot du Readme.md principal \(p.58\)](#)

Gestion des versions et organisation du code

Pour assurer une bonne gestion du code source, j'ai suivi les bonnes pratiques suivantes :

- **Commits nommés et explicites** : chaque modification a été accompagnée d'un **message de commit clair**, décrivant précisément l'action effectuée.
- **Création de branches spécifiques** pour chaque grande partie du projet afin de maintenir un historique propre et organisé.

Voir en annexe :

- [Annexe 26 : Screenshot de commits \(p.58\)](#)

Documentation technique par fonctionnalité

En complément du **README principal**, des **README spécifiques** ont été ajoutés à certaines parties du projet, notamment pour l'utilisation des fichiers SCSS/CSS et JavaScript.

Voir en annexe :

- [Annexe 27 : Screenshot du Readme.md concernant le style \(p.59\)](#)

Sécurité et maintenance du site

Au cours de notre formation, nous avons étudié les vulnérabilités les plus courantes en cybersécurité. En complément, nous avons eu accès à des ressources spécifiques sur la sécurisation des applications, notamment grâce à l'OWASP (Open Web Application Security Project), qui publie une liste des dix failles de sécurité les plus critiques.

Par ailleurs, Symfony intègre des outils pour renforcer la sécurité des applications, notamment des commandes permettant de vérifier la sécurité des dépendances et de gérer les mises à jour (`composer update`, `composer audit`, `symfony check:security`).

Mesures mises en place contre les principales failles de sécurité

Nous avons appliqué plusieurs bonnes pratiques pour protéger notre application contre les vulnérabilités suivantes :

Injection SQL

- Grâce à Doctrine ORM, toutes les requêtes sont automatiquement transformées en requêtes préparées, ce qui empêche l'injection SQL.
- Nous avons systématiquement utilisé le **QueryBuilder**, évitant ainsi la concaténation directe de données utilisateur dans les requêtes SQL.

Cross-Site Scripting (XSS)

- Le moteur de templates **Twig** échappe automatiquement les sorties HTML, JavaScript et autres contenus dynamiques, empêchant ainsi l'exécution de scripts malveillants.

Cross-Site Request Forgery (CSRF)

- Symfony intègre un système de protection CSRF via le composant **CsrfTokenManager**.
- Les formulaires générés par Symfony incluent automatiquement un **token CSRF**, garantissant que les requêtes ne puissent être envoyées que depuis une source légitime.

Inclusion de fichiers arbitraires

- Grâce à son **système d'autoloading et de routing sécurisé**, Symfony bloque l'inclusion non contrôlée de fichiers.

Exposition de données sensibles

- Les mots de passe des utilisateurs sont stockés de manière sécurisée grâce au **hachage bcrypt** (via `password_hash()` par défaut).

- Les informations sensibles (comme les clés API) sont stockées dans le fichier `.env`, évitant ainsi leur exposition directe dans le code.



Étant restés dans un environnement de développement, nous n'avons pas directement traité les vulnérabilités spécifiques à un environnement de production. Pour les autres failles de sécurité connues, nous avons pris soin de les répertorier dans le **Trello** du projet afin qu'elles puissent être traitées en priorité avant une mise en production.

Tests fonctionnels et unitaires

Nous avons effectué plusieurs tests fonctionnels et unitaires pour garantir la stabilité et la fiabilité des fonctionnalités, en utilisant notamment la classe **WebTestCase** de Symfony. Cette classe permet de simuler des interactions avec l'application web et de tester les routes, les contrôleurs et les formulaires de manière automatique. Voici un aperçu des tests réalisés :

Tests fonctionnels

- Test des routes du projet pour vérifier leur accessibilité.
- Test des routes avec un utilisateur connecté afin de valider les permissions.
- Vérification des liens présents sur la page d'accueil pour s'assurer qu'ils pointent vers les bonnes destinations.
- Test de la soumission du formulaire de connexion avec des informations valides.
- Test de la soumission du formulaire de connexion pour un utilisateur Admin.
- Vérification des redirections après la soumission des formulaires, selon les rôles et permissions.

- Test des messages d'erreur en cas de mauvaise information lors de la connexion (mauvais identifiant ou mot de passe).

Tests unitaires

- Test de la récupération des données du chatbot pour vérifier la bonne interaction avec l'API.

Voir en annexe :

- [Annexe 28 : Snippets de code : Tests \(p.60\)](#)
- [Annexe 29 : Snippets Terminal : test réussi / échoué \(p.61\)](#)

Nous n'avons pas testé toutes les fonctionnalités du projet, mais avons consigné dans **Trello** les tests à réaliser pour les développeurs à venir. Cela leur permettra de suivre les tests à effectuer et d'assurer que toutes les fonctionnalités seront validées avant la mise en production.

En conclusion

Travailler sur le projet Allo Afro Food a été enrichissant à plusieurs égards. D'abord, le fait de ne pas avoir de tuteur/tutrice développeur.se a été comme une lame à double tranchant : cela nous a laissé une grande liberté quant à la conception de A à Z mais nous avons également fait des choix qui n'auraient pas eu lieu d'être si nous avions eu un professionnel avec nous. J'ai de mon côté beaucoup appris en utilisant Symfony car c'est un outil très complet.

ANNEXES

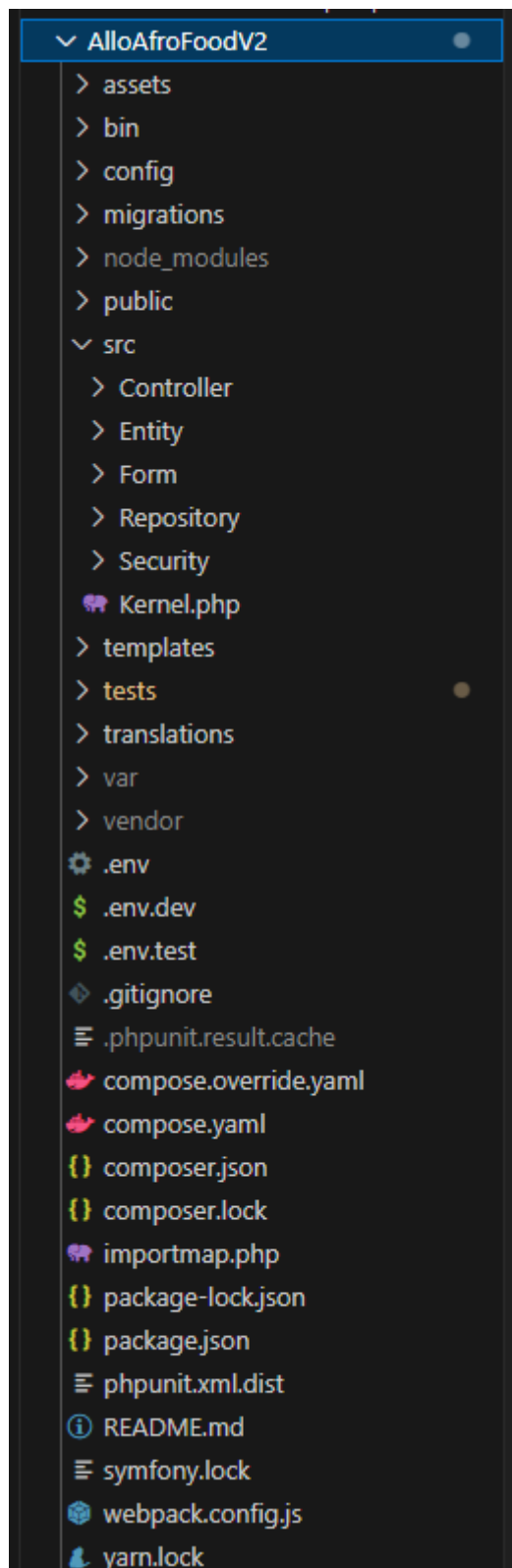
SOMMAIRE

SOMMAIRE.....	35
- Annexe 1 : Racine du projet (p.15).....	36
- Annexe 2 : Proposition de design (p.16).....	37
- Annexe 3 : Premières maquettes (p.16).....	38
- Annexe 4 : Maquettes réalisées avec Jude (p.16).....	39
- Annexe 5 : Snippet du fichier base.html.twig (p.17).....	40
- Annexe 6 : Homepage du site (p.19).....	41
- Annexe 7 : Snippet appStyle.scss (p.20).....	42
- Annexe 8 : Snippet webpack.config.js (p.20).....	43
- Annexe 8 : Snippet du block stylesheets (p.20).....	43
- Annexe 10 : Snippet du fichier security.yaml (p.21).....	44
- Annexe 11 : Snippet du fichier LoginAuthenticator (p.21).....	45
- Annexe 12 : Interfaces du back-office EasyAdmin (p.21).....	46
- Annexe 13 : Screenshot des messages de connexion (p.22).....	47
- Annexe 14 : Snippets application de Stripe (p.22).....	48
- Annexe 15 : Visuels du chatbot (p.24).....	49
- Annexe 16 : Représentation de la base de données (p.25).....	51
- Annexe 17 : Snippet du fichier .env (p.25).....	51
- Annexe 18 : Snippet d'un fichier de migration (p.26).....	52

- Annexe 19 : Snippet du controller de recherche & filtros (p.26).....	52
- Annexe 20 : Snippet du repository de recherche & filtros (p.26).....	53
- Annexe 21 : Screenshot de la pagination des produits (p.27).....	53
- Annexe 22 : Snippet du code du Menu repository (p.27).....	54
- Annexe 23 : Snippet du code du Menu controller (p.27).....	54
- Annexe 24 : Screenshot de la page Menu (p.27).....	55
- Annexe 25 : Screenshot du Readme.md (p.27).....	56
- Annexe 26 : Screenshot de commits (p.27).....	57
- Annexe 27 : Screenshot du Readme.md concernant les styles (p.27).....	58
- Annexe 28 : Snippet de code : Tests (p.27).....	59
- Annexe 29 : Snippet Terminal : test réussi / échoué (p.27).....	60

- Annexe 1 : Racine du projet (p.15)

Screenshot de la racine du projet Allo Afro Food.



← Les fichiers présents dans **assets** sont principalement le SCSS, le JSON et le JavaScript.

← Le dossier **migrations** comprend les fichiers contenant du SQL à travers duquel nous éditons notre base de données.

← Le dossier **public** comprend les fichiers compilés par Webpack Encore.

← **tests** comprend les tests fonctionnels et unitaires du projet.

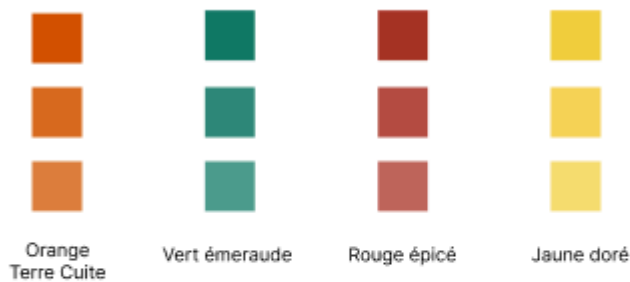
← Les fichiers **.env**, **.env.dev**, **.env.test** sont utilisés pour stocker les variables d'environnement (développement, production, test).

← **webpack.config.js** est le fichier de configuration de Webpack.

- Annexe 2 : Proposition de design (p.16)

D'après les demandes exprimées, premières propositions apportées sur Figma pour les couleurs, la typographies et les pictogrammes.

Couleur principale et ses teintes



Typographies

Bienvenue chez Allo Afro Food

Nous préparons des plats faits maisons pour toutes vos envies

BIENVENUE CHEZ ALLO AFRO FOOD

Nous préparons des plats faits maisons pour toutes vos envies

BIENVENUE CHEZ ALLO AFRO FOOD

Nous préparons des plats faits maisons pour toutes vos envies

BIENVENUE CHEZ ALLO AFRO FOOD

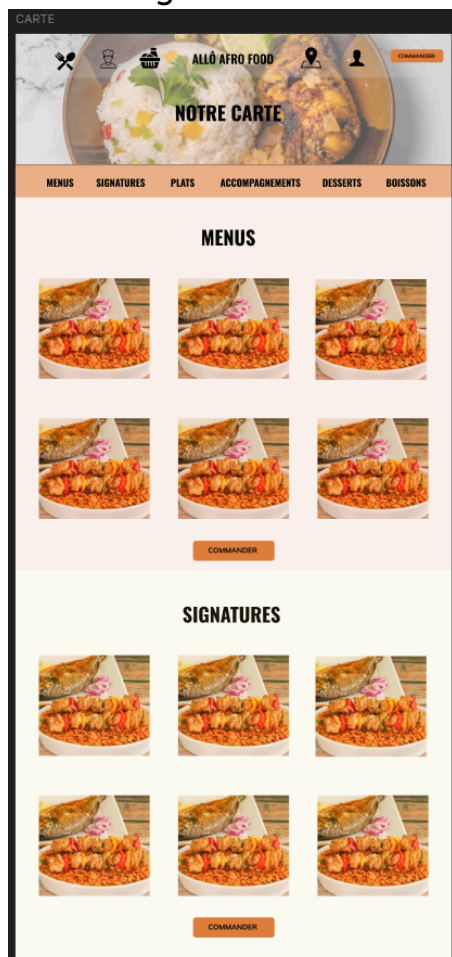
Nous préparons des plats faits maisons pour toutes vos envies



- Annexe 3 : Premières maquettes (p.16)

Premier jet pour le visuel du site, avant le développement et l'arrivée de Jude.

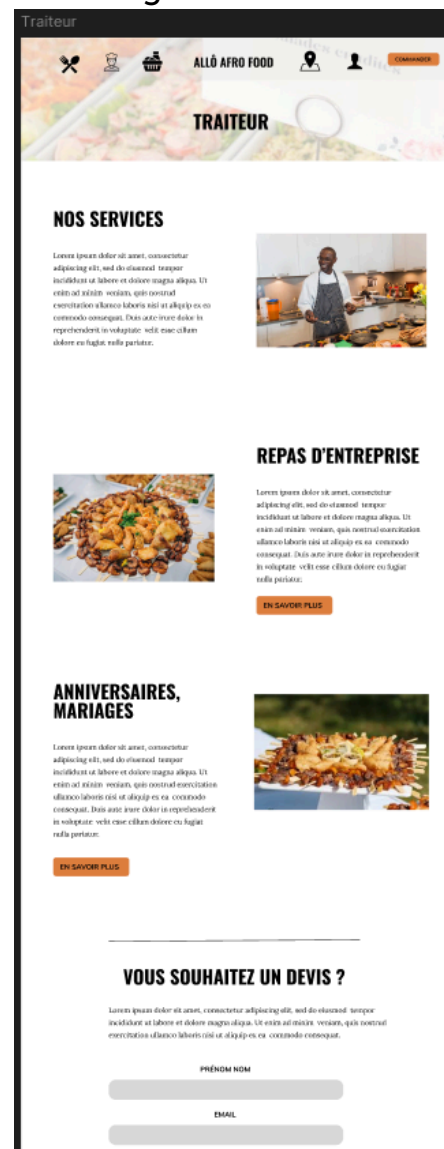
Page Notre Carte



Homepage

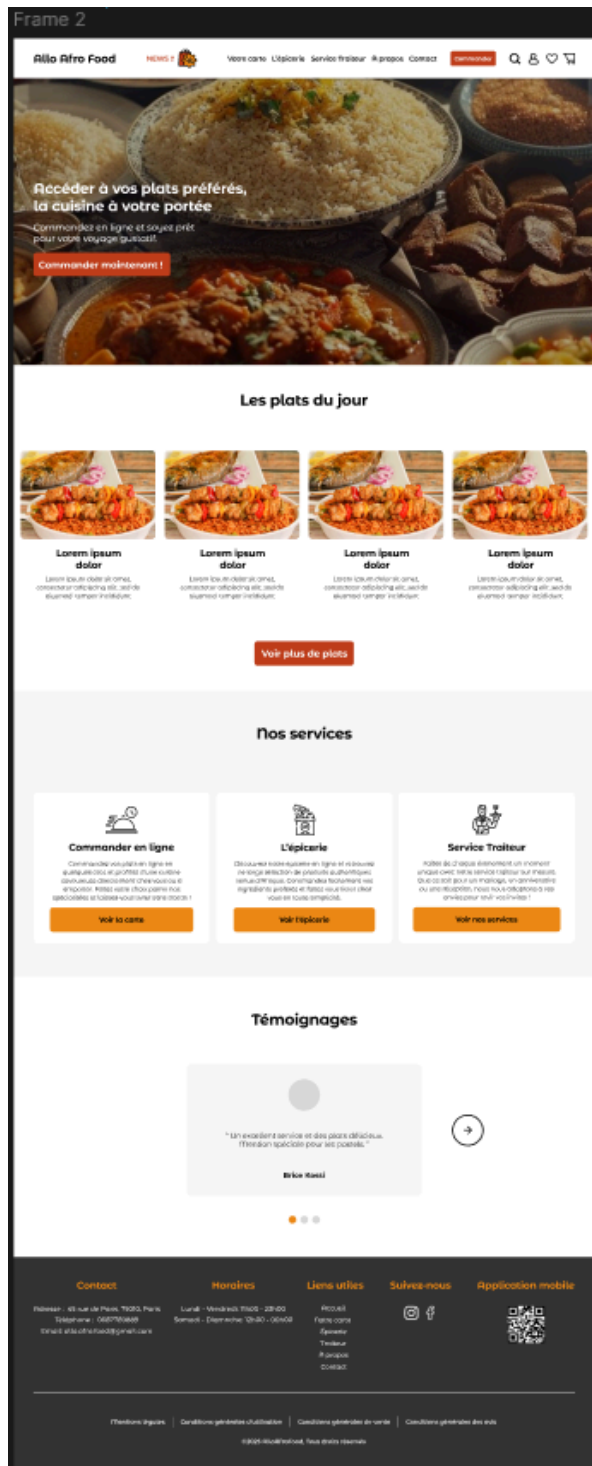


Page Traiteur

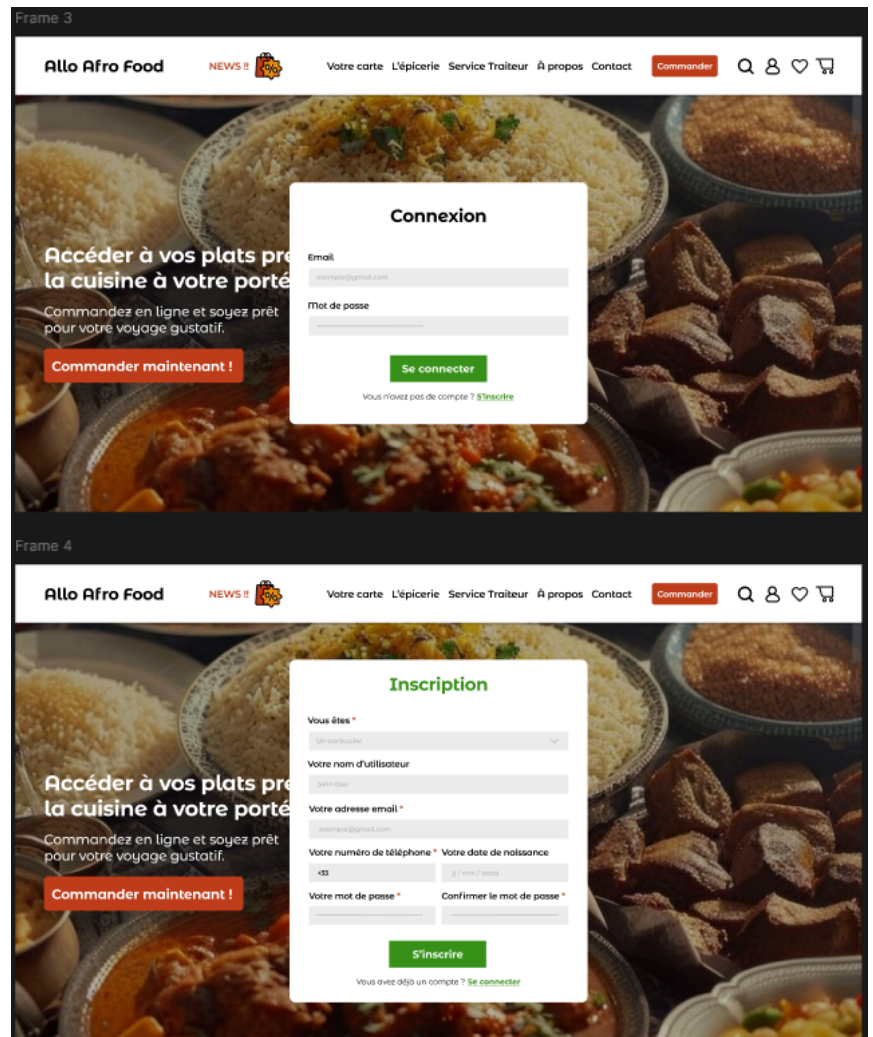


- Annexe 4 : Maquettes réalisées avec Jude (p.16)

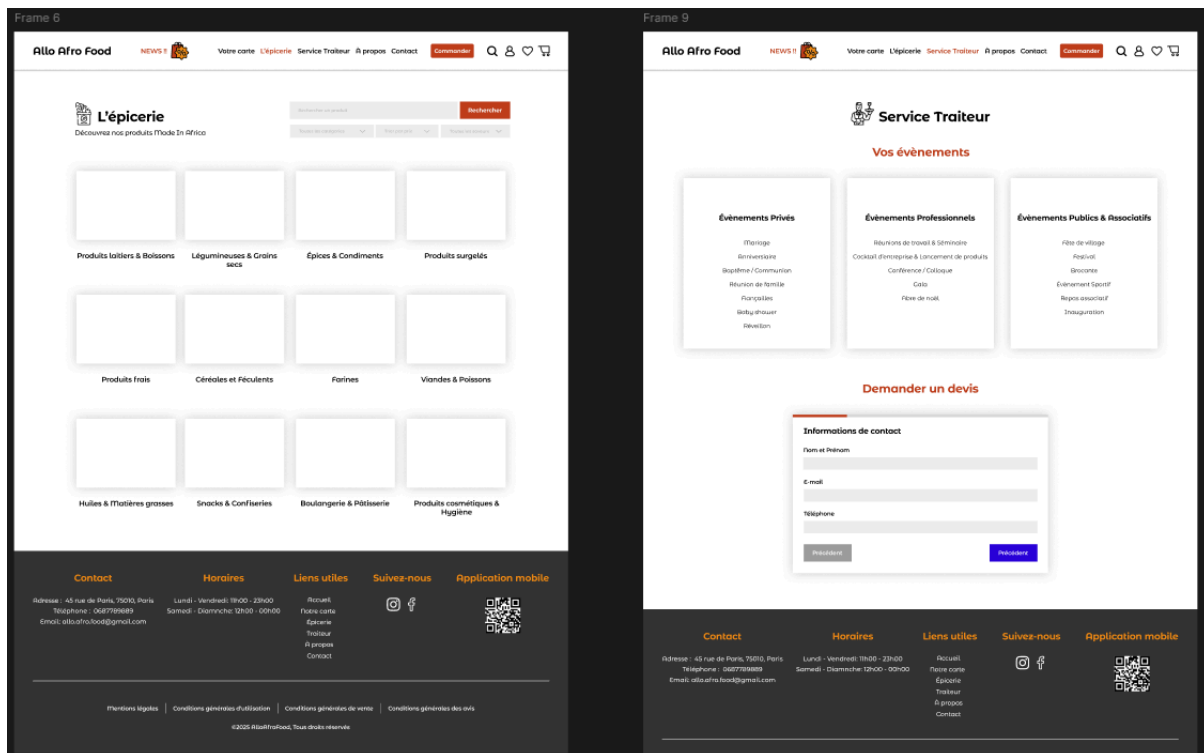
Homepage



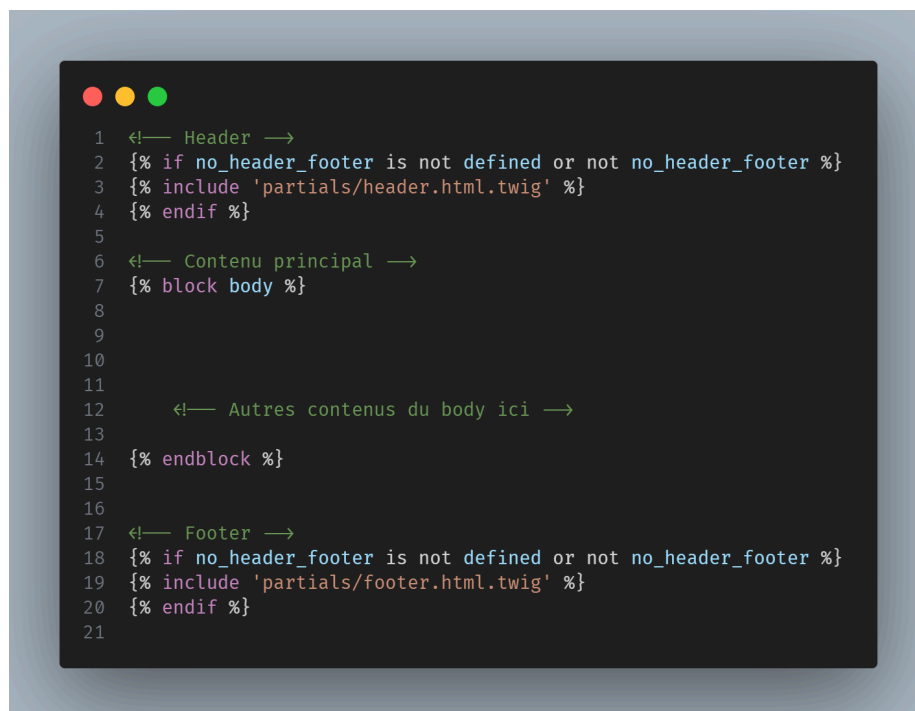
Homepage (avec modales)



Pages Épicerie & Traiteur

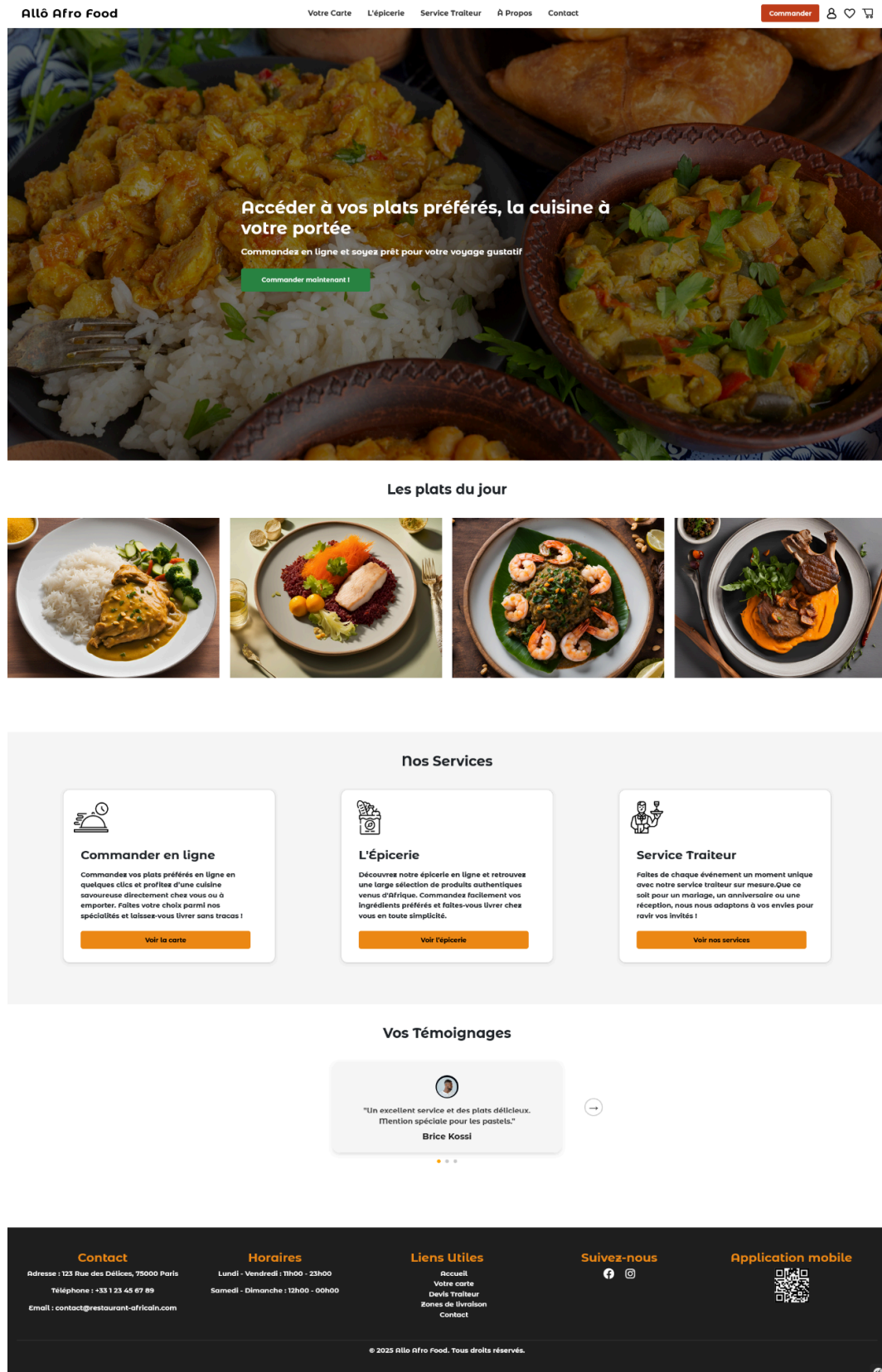


- Annexe 5 : Snippet du fichier base.html.twig (p.17)



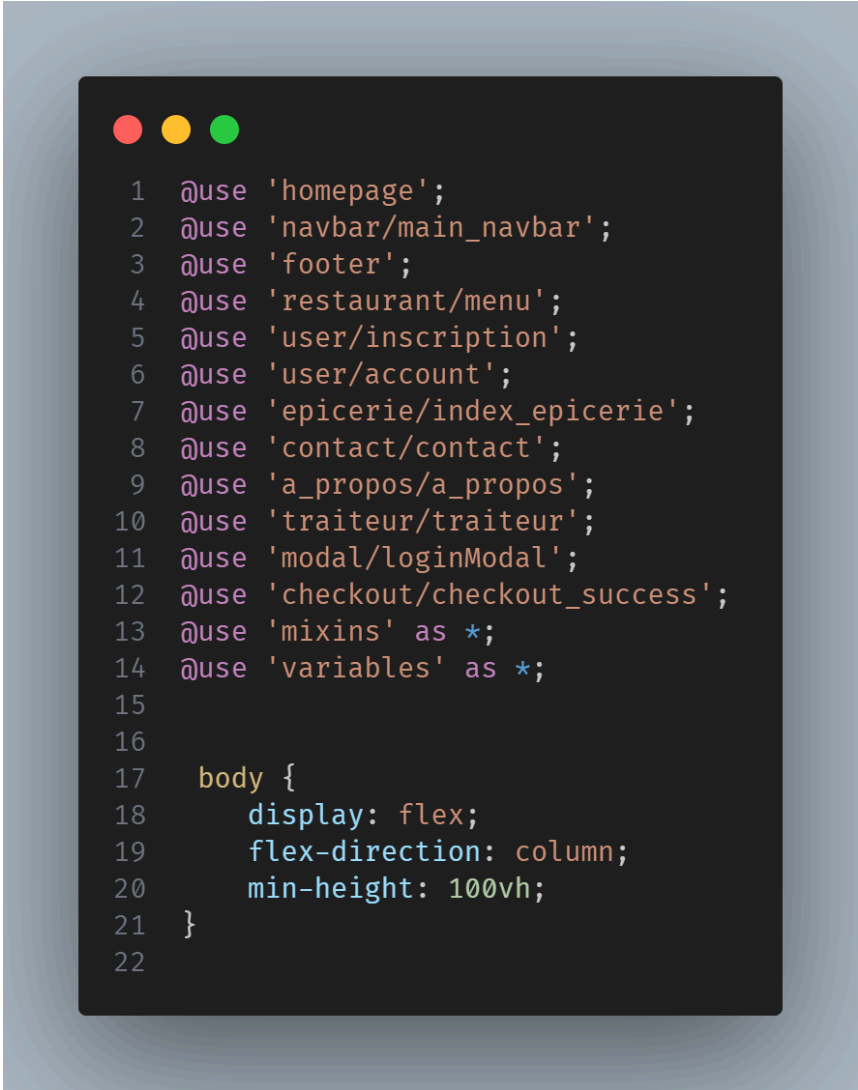
- Annexe 6 : Homepage du site (p.19)

L'image principale comprend 100% de la hauteur de l'écran (header compris).



- Annexe 7 : Snippet appStyle.scss (p.21)

Les autres fichiers SCSS sont importés dans le fichier avec le mot-clé `@use`.



```
1  @use 'homepage';
2  @use 'navbar/main_navbar';
3  @use 'footer';
4  @use 'restaurant/menu';
5  @use 'user/inscription';
6  @use 'user/account';
7  @use 'epicerie/index_epicerie';
8  @use 'contact/contact';
9  @use 'a_propos/a_propos';
10 @use 'traiteur/traiteur';
11 @use 'modal/loginModal';
12 @use 'checkout/checkout_success';
13 @use 'mixins' as *;
14 @use 'variables' as *;
15
16
17 body {
18     display: flex;
19     flex-direction: column;
20     min-height: 100vh;
21 }
22
```


- Annexe 8 : Snippet webpack.config.js (p.21)

On remarque que pour l'ajout de fichier de style dans Webpack en vue de la compilation, il y a 2 paramètres, le nom que l'on souhaite donner au fichier et son chemin.

```
1 .addStyleEntry('appStyle', './assets/styles/appStyle.scss')  
2
```

- Annexe 9 : Snippet du block stylesheets (p.21)

C'est donc dans nos fichiers Twig que l'on écrit le nom donné dans le fichier webpack.config.js.

```
1      <!-- Styles personnalisés -->  
2      {% block stylesheets %}  
3          {{ encore_entry_link_tags('appStyle') }}  
4      {% endblock %}  
5
```

- Annexe 10 : Snippet du fichier security.yaml (p.22)

Vous remarquerez qu'ici, on sécurise plusieurs choses comme l'algorithme de hachage des mots de passe, l'entité utilisée pour identifier un utilisateur sur le site, l'authentification personnalisée avec **LoginAuthenticator** et notamment le fait que seuls les utilisateurs ayant le rôle **ROLE_ADMIN** peuvent accéder à l'interface d'administration (URL **/admin**).

```
1 security:
2   # Définit l'algorithme de hachage des mots de passe pour les utilisateurs
3   password_hashers:
4     Symfony\Component\Security\Core\User\PasswordAuthenticatedUserInterface: 'auto'
5
6   # Définir le fournisseur d'utilisateurs utilisé pour charger un utilisateur par email
7   providers:
8     app_user_provider:
9       entity:
10        class: App\Entity\User
11        property: email
12
13  firewalls:
14    # Configuration du firewall pour l'environnement de développement (accès non sécurisé pour certaines routes)
15    dev:
16      pattern: ^/(_(profiler|wdt)|css|images|js)/
17      security: false
18
19    # Configuration du firewall principal avec authentification personnalisée
20    main:
21      lazy: true
22      provider: app_user_provider
23      custom_authenticator: App\Security>LoginAuthenticator
24      entry_point: App\Security>LoginAuthenticator
25      logout:
26        path: app_logout
27        # Définir la redirection après déconnexion
28        # target: app_any_route
29
30    remember_me:
31      secret: '%kernel.secret%'
32      lifetime: 604800 # Durée du cookie "remember me" en secondes (7 jours)
33      path: /
34      # Pour activer la fonctionnalité "se souvenir de moi" par défaut, décommentez la ligne suivante
35      # always_remember_me: true
36
37  # Contrôle d'accès pour les différentes routes
38  access_control:
39    # Seuls les utilisateurs avec le rôle ROLE_ADMIN peuvent accéder à l'interface d'administration
40    - { path: ^/admin, roles: ROLE_ADMIN }
41    # Exemple de contrôle d'accès pour les utilisateurs connectés (facultatif)
42    # - { path: ^/profile, roles: ROLE_USER }
43
```

- Annexe 11 : Snippet du fichier LoginAuthenticator (p.22)

Le fichier crée un **Passport** lors de l'authentification de l'utilisateur avec notamment un token CSRF. Il vérifie aussi que la requête envoyée pour la connexion est une requête **AJAX** (on l'utilise pour le formulaire présent dans la modal de connexion).

```
1 // Cette méthode est appelée lors de l'authentification pour récupérer les informations de l'utilisateur
2 public function authenticate(Request $request): Passport
3 {
4     // Récupère l'email et le mot de passe de la requête
5     $email = $request->request->get('email');
6
7     // Enregistre l'email dans la session pour l'affichage de la dernière adresse email utilisée
8     $request->getSession()->set(SecurityRequestAttributes::LAST_USERNAME, $email);
9
10    // Retourne un Passport qui contient les informations nécessaires à l'authentification (badge utilisateur, credentials, etc.)
11    return new Passport(
12        new UserBadge($email), // Le UserBadge contient l'email de l'utilisateur pour l'authentification
13        new PasswordCredentials($request->request->get('password')), // Les informations d'identification (mot de passe)
14        [
15            new CsrfTokenBadge('authenticate', $request->request->get('_csrf_token')), // CSRF Token pour la sécurité
16            new RememberMeBadge(), // Gère la fonctionnalité "se souvenir de moi"
17        ]
18    );
19 }
20
21 // Cette méthode est appelée lorsque l'authentification réussit
22 public function onAuthenticationSuccess(Request $request, TokenInterface $token, string $firewallName): ?Response
23 {
24     // Vérifie si la requête est une requête AJAX
25     if ($request->isXmlHttpRequest()) {
26         // Si c'est une requête AJAX, on renvoie du JSON avec l'URL de redirection
27         $targetPath = $this->getTargetPath($request->getSession(), $firewallName);
28         if ($targetPath) {
29             return new JsonResponse([
30                 'status' => 'success',
31                 'redirect' => $targetPath, // Redirige vers la dernière page visitée
32                 'message' => 'Bienvenue, ' . $token->getUser()->getUserIdentifier() . ' !'
33             ]);
34         }
35
36         // Récupère les rôles de l'utilisateur pour définir où le rediriger
37         $roles = $token->getRoleNames(); // Récupère les rôles de l'utilisateur
38
39         // Redirige vers l'interface d'administration si l'utilisateur est un admin, sinon vers la page d'accueil
40         $redirectUrl = in_array('ROLE_ADMIN', $roles, true)
41             ? $this->urlGenerator->generate('admin')
42             : $this->urlGenerator->generate('homepage');
43
44         return new JsonResponse([
45             'status' => 'success',
46             'redirect' => $redirectUrl,
47             'message' => 'Vous êtes bien connecté(e), bienvenue !'
48         ]);
49     }
50
51     // Si ce n'est pas une requête AJAX, redirige normalement
52     $targetPath = $this->getTargetPath($request->getSession(), $firewallName);
53     if ($targetPath) {
54         return new RedirectResponse($targetPath); // Redirige vers la dernière page visitée
55     }
56
57     // Redirige en fonction des rôles de l'utilisateur
58     $roles = $token->getRoleNames();
59     $redirectUrl = in_array('ROLE_ADMIN', $roles, true)
60         ? $this->urlGenerator->generate('admin') // Redirection vers l'admin si rôle admin
61         : $this->urlGenerator->generate('homepage'); // Redirection vers la page d'accueil si autre rôle
62
63     return new RedirectResponse($redirectUrl);
64 }
65 }
```

- Annexe 12 : Interfaces du back-office EasyAdmin (p.22)

Gestion des utilisateurs du site

Allô Afro Food

- Tableau de bord
- Utilisateurs
- Commandes
- Produits Restaurant
- Produits Epicerie
- Catégories Produits Epicerie
- Services Traiteur

Rechercher

Utilisateurs

☐	Prénom	Nom	E-mail	Numéro de téléphone	Date d'anniversaire	Type d'utilisateur	
☐	Admin	AAF	mikesylla@gmail.com	Aucun(e)	1 janv. 1905	a:0:[]	...
☐	Jane	Doe	jane.doe@gmail.com	0678787878	9 juil. 1950	a:0:[]	...
☐	Jack	Doe	jack.doe@gmail.com	0656565656	2 avr. 1926	Particulier	...
☐	Anne	Doe	anne.doe@gmail.com	0787898789	1 févr. 1984	Un particulier	...
☐	Joel	Doe	joel.doe@gmail.com	0721232123	2 janv. 1969	Entreprise	...
☐	Vic	Doe	vic.doe@gmail.com	0787878787	Aucun(e)	Particulier	...
☐	Vik	Doe	vik.doe@gmail.com	0787878787	Aucun(e)	Particulier	...
☐	Nico	Doe	nico.doe@gmail.com	0787878786	Aucun(e)	Particulier	...
☐	Pat	Doe	pat.doe@gmail.com	0787878787	Aucun(e)	Particulier	...
☐	Jasper	Doe	jasper.doe@gmail.com	07878787	Aucun(e)	Particulier	...
☐	Louise	Doe	louise.doe@gmail.com	0787878787	Aucun(e)	Particulier	...
☐	Jimbo	Doe	jimbo.doe@gmail.com	Aucun(e)	Aucun(e)	Particulier	...
☐	Samuel	Doe	samuel.doe@gmail.com	0684848484	Aucun(e)	Particulier	...
☐	Isabelle	Doe	isa.doe@gmail.com	0787878787	Aucun(e)	Particulier	...
☐	Test	Doe	test.doe@gmail.com	787878787	Aucun(e)	Particulier	...
☐	Test2	Doe	test2.doe@gmail.com	0789898989	Aucun(e)	Particulier	...

16 résultats

Gestion des produits du restaurant

Allô Afro Food

- Tableau de bord
- Utilisateurs
- Commandes
- Produits Restaurant
- Produits Epicerie
- Catégories Produits Epicerie
- Services Traiteur

Rechercher

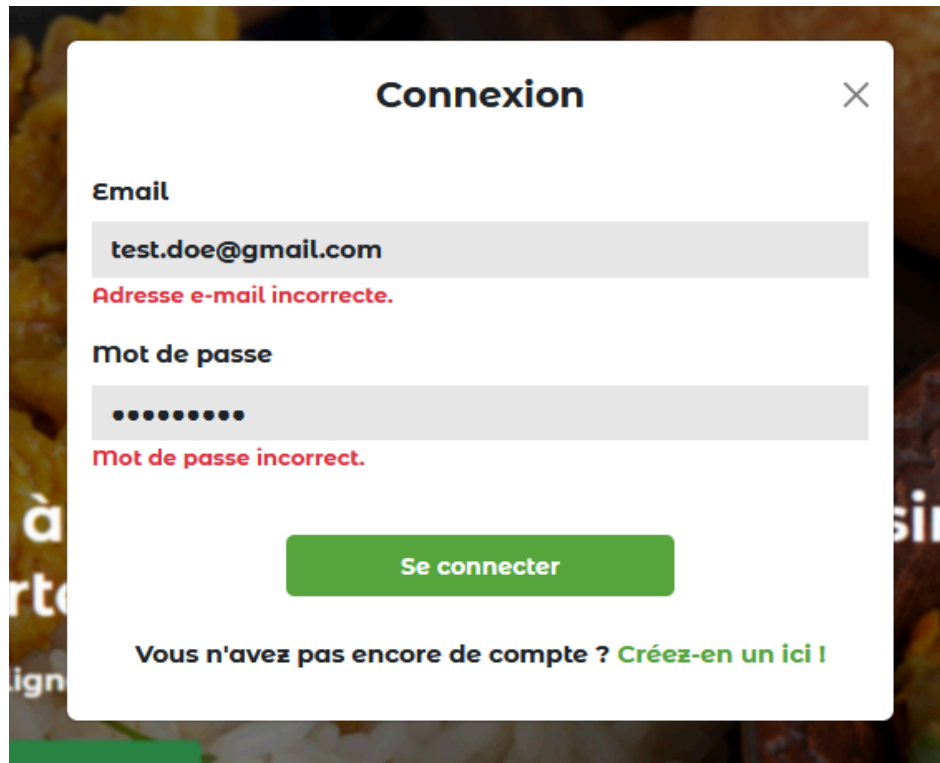
Produit Restaurant

Créer un nouveau produit

☐	Nom du produit	Catégorie du produit	Plat du jour	Prix du produit	Disponible	Jours de disponibilité	Description du produit	Image du produit	Date de création	Dernière mise à jour	
☐	Pastel au thon	Pastel	☒	3.00 €	☒	du Lundi au jeudi	Pastel au thon fait maison servi avec une délicieuse sauce tomate.		10 févr. 2025	12 févr. 2025	...
☐	Accras de morue au piment d'Espelette et bissap	Entrée	☒	4.00 €	☒	Tous les jours	Beignets de morue moelleux, parfumés au piment et servis avec une sauce à la fleur d'hibiscus.		19 févr. 2025	Aucun(e)	...
☐	Brochettes de gésiers confits au poivre de Penja	Entrée	☒	4.50 €	☒	Tous les jours	Gésiers marinés et grillés, relevés au poivre camerounais et accompagnés d'une vinaigrette citron-moutarde.		19 févr. 2025	Aucun(e)	...
☐	Aloco croustillant et sa sauce épicée	Entrée	☒	3.50 €	☒	Tous les jours	Rondelles de banane plantain frites, servies avec une sauce tomate-gingembre légèrement pimentée.		19 févr. 2025	Aucun(e)	...
☐	Soupe arachide aux crevettes	Entrée	☒	4.00 €	☒	Tous les jours	Velouté de pâte d'arachide avec des crevettes sautées et des éclats de cacahuètes.		19 févr. 2025	Aucun(e)	...
☐	Salade de fonio et mangue verte	Entrée	☒	3.50 €	☒	Tous les jours	Mélange frais de fonio, mangue verte râpée, menthe fraîche et vinaigrette au citron vert.		19 févr. 2025	Aucun(e)	...

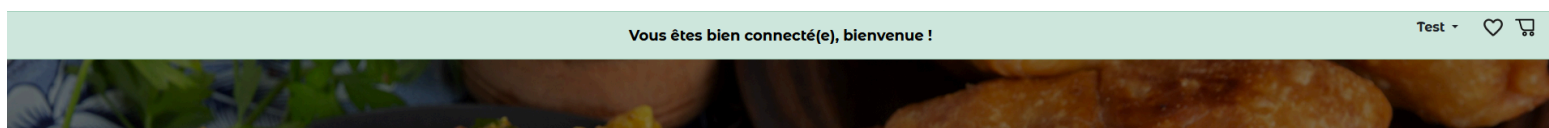
- Annexe 13 : Screenshot des messages de connexion (p.23)

Messages d'erreur générique (échec)



The screenshot shows a login modal window titled "Connexion" with a close button (X) in the top right corner. It contains two input fields: "Email" and "Mot de passe". The email field contains "test.doe@gmail.com" and has a red error message "Adresse e-mail incorrecte." below it. The password field is masked with dots and has a red error message "Mot de passe incorrect." below it. A green "Se connecter" button is centered below the fields. At the bottom, there is a link: "Vous n'avez pas encore de compte ? [Créez-en un ici !](#)".

Message de bienvenue (réussite)



- Annexe 14 : Snippets application de Stripe (p.23)

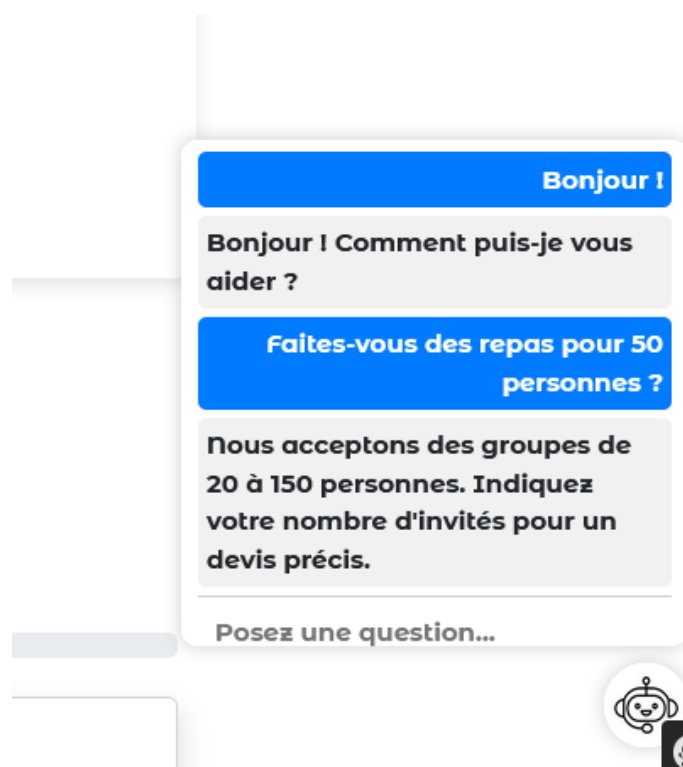
Dans le fichier JavaScript suivant, on remarque la création du `PaymentMethod` de Stripe, décrivant les moyens de paiement utilisés par l'utilisateur. Les données sont ensuite envoyées avec `fetch()` au serveur Symfony.

```
1 // Création du PaymentMethod
2 const { paymentMethod, error } = await stripe.createPaymentMethod({
3   type: "card",
4   card: cardNumber, // Ici, on utilise `cardNumber` (pas `cardElement`)
5   billing_details: {
6     name: cardHolderName,
7   },
8 });
9
10 if (error) {
11   console.error(error.message);
12   alert("Erreur lors de la création du paiement : " + error.message);
13   return;
14 }
15
16 // Envoi du PaymentMethodId au serveur Symfony
17 const response = await fetch("/process-payment", {
18   method: "POST",
19   headers: { "Content-Type": "application/json" },
20   body: JSON.stringify({
21     paymentMethodId: paymentMethod.id,
22     totalAmount: totalAmountInCents,
23     cart: cart,
24     deliveryType: deliveryType,
25   }),
26 });
27
28 const paymentResult = await response.json();
29
30 if (paymentResult.success) {
31   // Vérifie si une authentification est nécessaire (3D Secure)
32   const { paymentIntent, error } = await stripe.confirmCardPayment(
33     paymentResult.clientSecret,
34     {
35       return_url: `http://localhost:8000/checkout/success?orderId=${paymentResult.orderId}`,
36     }
37   );
38 }
39
40 if (error) {
41   console.error(error.message);
42   alert("Échec du paiement : " + error.message);
43   window.location.href = "/checkout/error";
44 } else if (paymentIntent.status === "succeeded") {
45   // ✅ Vider le localStorage
46   localStorage.removeItem("panier");
47   window.location.href = `http://localhost:8000/checkout/success?orderId=${paymentResult.orderId}`;
48 }
49 } else {
50   alert("Erreur serveur : " + paymentResult.error);
51   window.location.href = "/checkout/error";
52 }
53
```

Le back-end reçoit les données du front-end et les extrait. Il crée ensuite un **PaymentIntent** de Stripe qui se charge du paiement. Il renvoie ensuite les données au front-end pour soit une vérification supplémentaire soit une redirection.

```
1 // Récupérer les données envoyées depuis le frontend
2 $data = json_decode($request->getContent(), true);
3
4 // Vérifier si le JSON est valide
5 if ($data === null) {
6     return $this->json(['success' => false, 'error' => 'Le format des données envoyées est invalide']);
7 }
8
9 // Extraire les données
10 $totalAmount = $data['totalAmount'] ?? null;
11 $paymentMethodId = $data['paymentMethodId'] ?? null;
12 $deliveryType = $data['deliveryType'] ?? null;
13 $address = $data['address'] ?? null;
14 $cart = $data['cart'] ?? [];
15
16 // Vérifier si les données nécessaires sont présentes
17 if (!$paymentMethodId || !$totalAmount || !$cart) {
18     return $this->json(['success' => false, 'error' => 'Données manquantes']);
19 }
20
21 try {
22     // Créer un PaymentIntent avec Stripe
23     $paymentIntent = PaymentIntent::create([
24         'amount' => $totalAmount,
25         'currency' => 'eur',
26         'payment_method' => $paymentMethodId,
27         'confirmation_method' => 'manual', // Mode manuel pour 3D Secure
28         'confirm' => true, // Confirmation immédiate
29         'return_url' => 'http://localhost:8000/checkout/success', // URL de redirection après paiement réussi
30     ]);
31 }
```

- Annexe 15 : Visuels du chatbot (p.25)

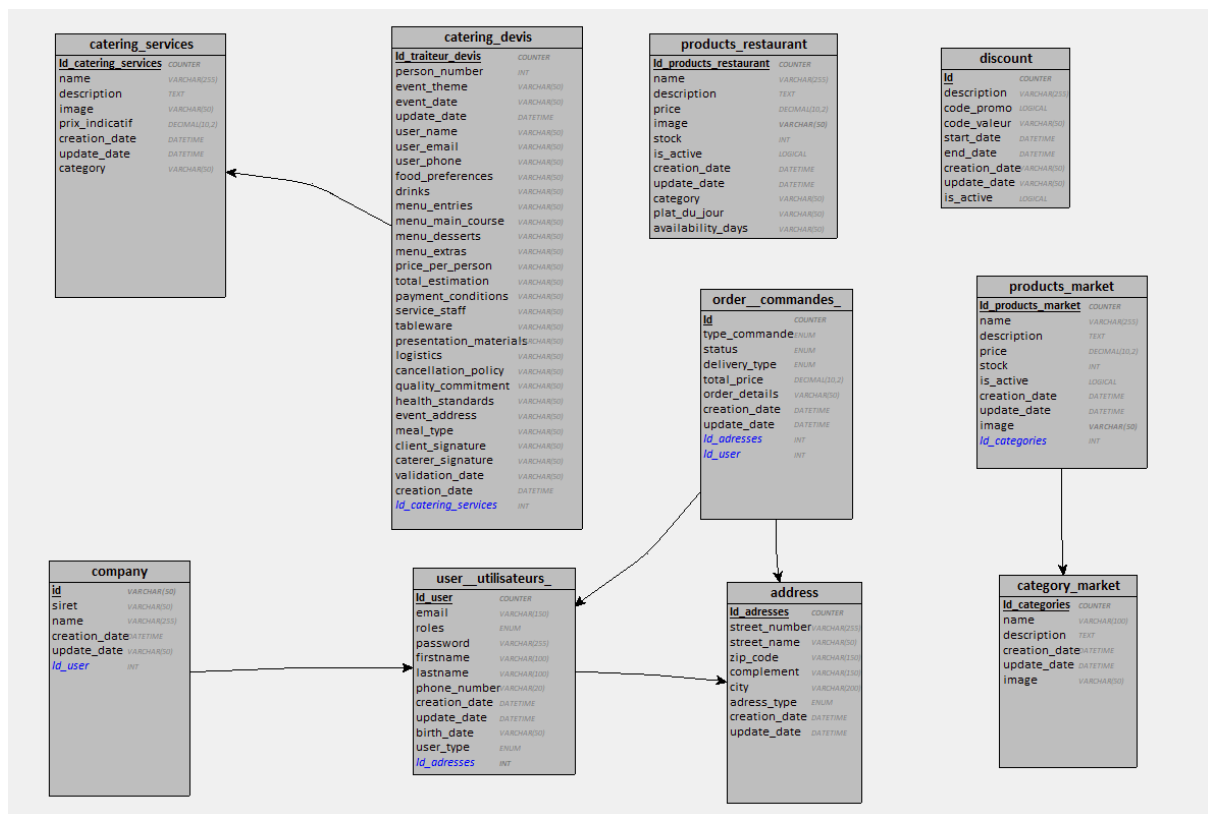


Ci-dessous le fichier JSON qui stocke des paires clés - valeurs du chatbot. Chaque mot possède des synonymes (clé "keywords") et une réponse apportée par le chatbot (clé "response").

```
1 {
2
3   "bonjour": {
4     "keywords": ["bonjour", "salut", "coucou", "hello", "bonsoir"],
5     "response": "Bonjour ! Comment puis-je vous aider ?"
6   },
7   "devis": {
8     "keywords": ["devis", "tarif", "prix", "facture", "commande"],
9     "response": "Pour un devis personnalisé, veuillez remplir notre formulaire : /devis."
10  },
11  "date": {
12    "keywords": ["date", "disponibilité", "calendrier", "réserver", "quand"],
13    "response": "Nous recommandons de réserver au moins une semaine à l'avance. Pour vérifier les disponibilités, contactez-nous."
14  },
15  "nombre_personnes": {
16    "keywords": ["personnes", "invités", "combien", "minimum", "maximum", "taille", "capacité"],
17    "response": "Nous acceptons des groupes de 20 à 150 personnes. Indiquez votre nombre d'invités pour un devis précis."
18  },
19  "mariage": {
20    "keywords": ["mariage", "noces", "cérémonie", "réception"],
21    "response": "Nous proposons des services traiteur pour les mariages. Contactez-nous pour un devis."
22  },
23  "anniversaire": {
24    "keywords": ["anniversaire", "fête", "birthday", "bougie"],
25    "response": "Organisez votre anniversaire avec nous ! Découvrez nos offres spéciales."
26  },
27  "bapteme": {
28    "keywords": ["baptême", "naissance", "célébration", "bébé"],
29    "response": "Nous pouvons organiser un traiteur pour votre baptême. Contactez-nous pour plus d'infos."
30  },
31  "fetes_religieuses": {
32    "keywords": ["Noël", "Ramadan", "Pâques", "Aid", "Hanouka", "religieux", "fête sacrée"],
33    "response": "Nous proposons des menus adaptés aux fêtes religieuses. Demandez-nous plus de détails."
34  },
35  "soiree_privée": {
36    "keywords": ["soirée", "privé", "vip", "événement", "cocktail"],
37    "response": "Nous organisons des soirées privées pour 20 à 150 personnes. Faites-nous part de vos besoins."
38  },
39  "soiree_entreprise": {
40    "keywords": ["entreprise", "corporate", "team building", "séminaire", "réunion"],
41    "response": "Pour une soirée d'entreprise, nous avons plusieurs formules adaptées. Contactez-nous !"
42  },
43 }
```


- Annexe 16 : Représentation de la base de données (p.25)

Notre modèle a évolué au fil du temps, l'image ci-dessous représente l'état de la base de données à la fin du stage.



- Annexe 17 : Snippet du fichier .env (p.25)

La ligne de code ci-dessous configure le projet avec la base de données hébergée sur Hostinger, on y retrouve :

- Le système de gestion de base de données (mysql).
- L'identifiant et le mot de passe (u347722888_admin_alloafro + mot rayé).
- Le serveur pointant vers la base (srv1340.hstgr.io:3306).

```
1 DATABASE_URL="mysql://u347722888_admin_alloafro: [mot rayé]@srv1340.hstgr.io:3306/u347722888_alloafrofood?serverVersion=8.0.326charset=utf8mb4"
2
```

- Annexe 18 : Snippet d'un fichier de migration (p.27)

```
Tabnine | Edit | Test | Explain | Document | 0 references
public function up(Schema $schema): void
{
    // this up() migration is auto-generated, please modify it to your needs
    $this->addSql('ALTER TABLE address CHANGE street_number street_number VARCHAR(50) DE
    $this->addSql('ALTER TABLE category_market CHANGE creation_date creation_date DATETI
    $this->addSql('ALTER TABLE catering_devis CHANGE creation_date creation_date DATETIM
    $this->addSql('ALTER TABLE catering_services CHANGE creation_date creation_date DATE
    $this->addSql('ALTER TABLE company CHANGE siret siret VARCHAR(14) DEFAULT NULL, CHAN
    $this->addSql('ALTER TABLE discount CHANGE creation_date creation_date DATETIME DEFA
    $this->addSql('ALTER TABLE `order` DROP check_date, CHANGE creation_date creation_da
    $this->addSql('ALTER TABLE products_market CHANGE creation_date creation_date DATETI
    $this->addSql('ALTER TABLE products_restaurant CHANGE availability_days availability
    $this->addSql('ALTER TABLE user CHANGE roles roles JSON NOT NULL, CHANGE phone_numbe
    $this->addSql('ALTER TABLE messenger_messages CHANGE delivered_at delivered_at DATET
}

Tabnine | Edit | Test | Explain | Document | 0 references
public function down(Schema $schema): void
{
    // this down() migration is auto-generated, please modify it to your needs
    $this->addSql('ALTER TABLE discount CHANGE creation_date creation_date DATETIME DEFA
    $this->addSql('ALTER TABLE company CHANGE siret siret VARCHAR(14) DEFAULT \'NULL\'
    $this->addSql('ALTER TABLE address CHANGE street_number street_number VARCHAR(50) DE
    $this->addSql('ALTER TABLE catering_services CHANGE creation_date creation_date DATE
    $this->addSql('ALTER TABLE products_market CHANGE creation_date creation_date DATETI
    $this->addSql('ALTER TABLE category_market CHANGE creation_date creation_date DATETI
    $this->addSql('ALTER TABLE products_restaurant CHANGE availability_days availability
    $this->addSql('ALTER TABLE `order` ADD check_date DATETIME NOT NULL COMMENT \'(DC2T)
    $this->addSql('ALTER TABLE catering_devis CHANGE price_per_person price_per_person N
    $this->addSql('ALTER TABLE user CHANGE roles roles LONGTEXT NOT NULL COLLATE `utf8mb
    $this->addSql('ALTER TABLE messenger_messages CHANGE delivered_at delivered_at DATET
}
```

- Annexe 19 : Snippet du controller de recherche & filtres (p.27)

```
1 #Route('/recherche', name: 'recherche')]
2 public function rechercherProduits(Request $request, ProductsMarketRepository $prodMarket): Response
3 {
4     // Récupération de toutes les catégories pour afficher les options du filtre
5     $categories = $this->entityManager->getRepository(CategoryMarket::class)->findAll();
6
7     // Récupération des paramètres de recherche envoyés via la requête GET
8     $search = $request->query->get('search');
9     $category = $request->query->get('category');
10    $priceOrder = $request->query->get('price_order');
11
12    // Appliquer les filtres à la requête
13    $products = $prodMarket->findByFilters($search, $category, $priceOrder);
14
15    // Retourner la vue avec les produits filtrés et les critères sélectionnés
16    return $this->render('epicerie/recherche.html.twig', [
17        'categories' => $categories,
18        'products' => $products,
19        'search' => $search,
20        'category' => $category,
21        'price_order' => $priceOrder,
22    ]);
23 }
24
```

- Annexe 20 : Snippet du repository de recherche & filtres (p.28)

```

1  public function findByFilters($search = null, $category = null, $priceOrder = null)
2  {
3      // Créer une requête de sélection des produits avec QueryBuilder
4      $qb = $this->createQueryBuilder('p');
5
6      // Filtrer par catégorie si une catégorie est sélectionnée
7      if ($category) {
8          $qb->andWhere('p.category = :category')
9              ->setParameter('category', $category);
10     }
11
12     // Filtrer par mot-clé dans le nom ou la description des produits
13     if ($search) {
14         $qb->andWhere('p.name LIKE :search OR p.description LIKE :search')
15             ->setParameter('search', '%'.$search.'%');
16     }
17
18     // Appliquer le tri par prix si un ordre est défini ('asc' ou 'desc')
19     if ($priceOrder) {
20         $qb->orderBy('p.price', $priceOrder);
21     }
22
23     // Exécuter la requête et retourner les résultats
24     return $qb->getQuery()->getResult();
25 }
26

```

- Annexe 21 : Screenshot de la pagination des produits (p.28)



1 2

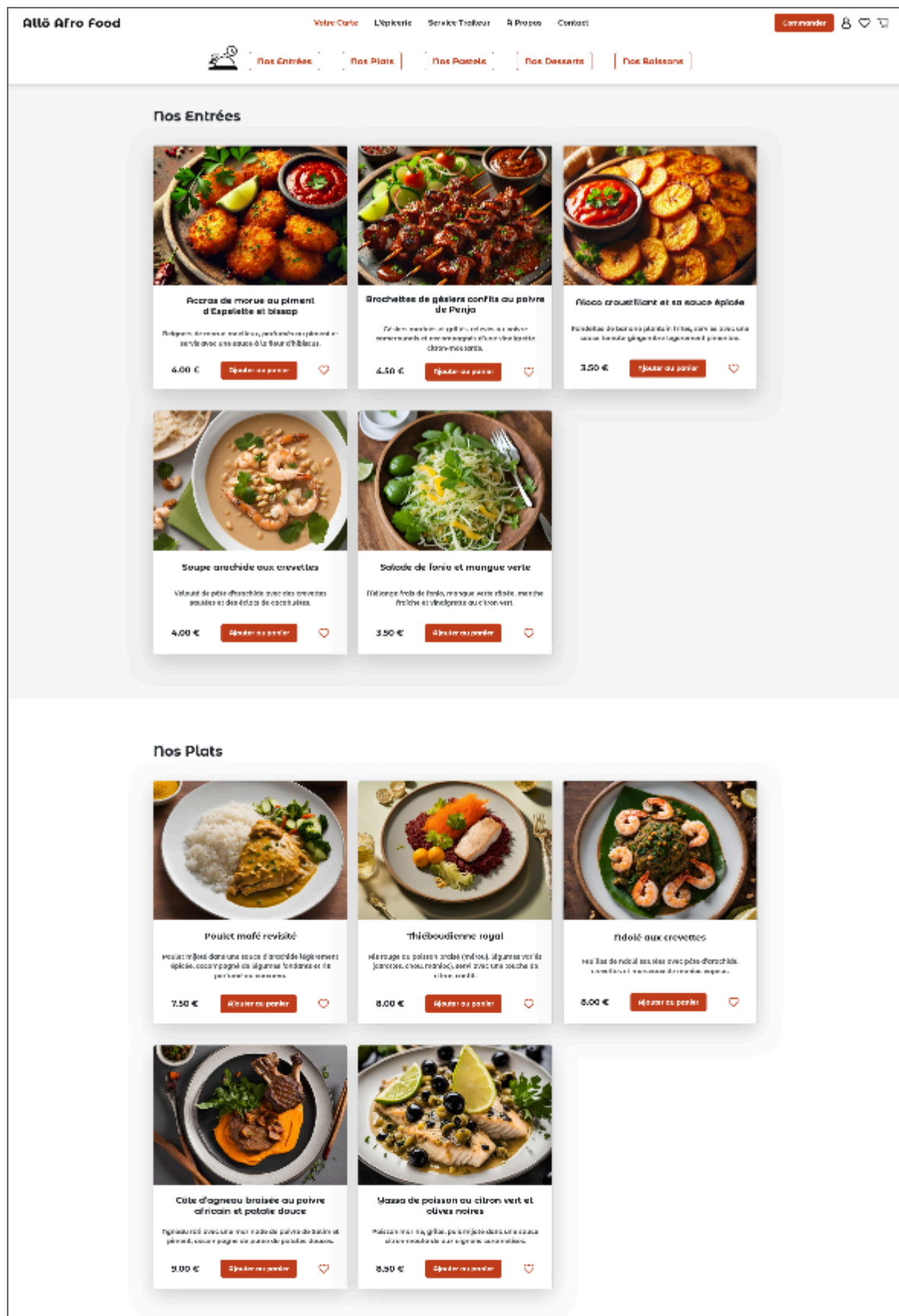
- Annexe 22 : Snippet du code du Menu repository (p.29)

```
1 public function findByCategory(string $category): array
2 {
3     return $this->createQueryBuilder('p')
4         ->where('p.category = :category')
5         ->setParameter('category', $category)
6         ->getQuery()
7         ->getResult();
8 }
9
```

- Annexe 23 : Snippet du code du Menu controller (p.29)

```
1 #[Route('/menu', name: 'menu')]
2 public function index(ProductsRestaurantRepository $productsRes): Response
3 {
4
5     return $this->render('menu_restaurant/index.html.twig', [
6         'menus' => $productsRes->findByCategory('menu'),
7         'entrees' => $productsRes->findByCategory('entree'),
8         'plats' => $productsRes->findByCategory('plat'),
9         'pastels' => $productsRes->findByCategory('pastel'),
10        'accompagnements' => $productsRes->findByCategory('accompagnement'),
11        'desserts' => $productsRes->findByCategory('dessert'),
12        'boissons' => $productsRes->findByCategory('boisson'),
13    ]);
14 }
```

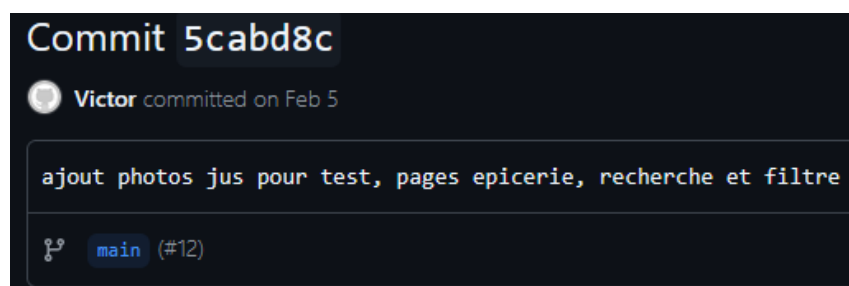
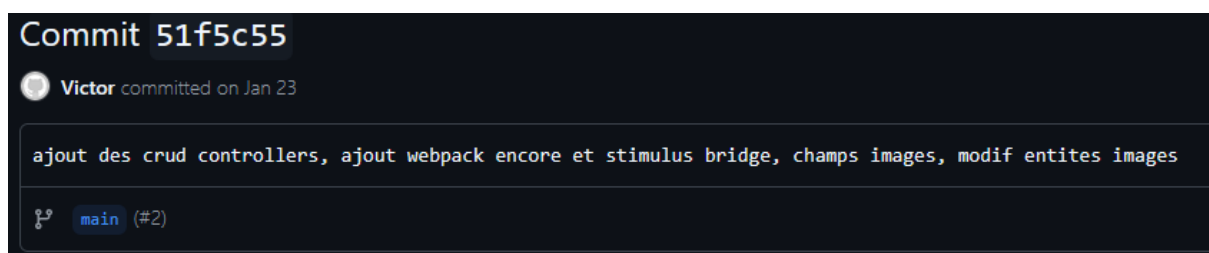
- Annexe 24 : Screenshot de la page Menu (p.29)



- Annexe 25 : Screenshot du Readme.md (p.30)



- Annexe 26 : Screenshot de commits (p.30)



- Annexe 27 : Screenshot du Readme.md concernant les styles (p.27)

Infos sur les styles

Comment ça marche ?

Grâce à Webpack Encore, tous les fichiers SCSS, sont compilés en un seul fichier CSS global. On importe les fichiers SCSS partiels (commençant par _) dans appStyle.scss et ce dernier est compilé par Webpack Encore (voir webpack.config.js à la racine du projet), en sortie, on retrouve appStyle.css dans le dossier public/build.

On a essayé de donner des noms de classe/id aux éléments HTML selon leur page. Donc par exemple en plus de la classe "container" pour une div, on y ajoute une classe "menu_container" pour personnaliser cette div dans la page menu. Avec ce nommage, on est sûr que les styles ne rentreront pas en conflit avec d'autres lors du compilation. Le SCSS est aussi pratique : le "nesting" permet de ne pas renommer toutes les balises car un h2 imbriqué dans un menu_container n'affectera pas les autres h2 par exemple.

On a essayé de donner des titres assez explicites aux dossiers de telle sorte qu'ils collent avec les pages twig du site (à améliorer). Pour plus de clarté voici les pages du site et leur dossiers styles respectifs :

- La Home Page (homepage)
- Votre Carte (restaurant)
- L'Épicerie (epicerie)
- Service Traiteur (traiteur)
- À Propos (a_propos)
- Contact (contact)
- Le style du compte utilisateur et de la page Inscription se trouvent dans le dossier user.
- Le style du Header se trouve dans le dossier navbar.
- Le style de la modal de connexion se trouve dans le dossier modal.
- Le style du footer n'est pas dans un dossier.

On a créé un fichier mixins et variables pour le site, mais les variables ne sont pas utilisées (à vous de jouer).

On utilise aussi Bootstrap pour le projet, vous retrouverez plein de classes bs dans le HTML.

- Annexe 28 : Snippet de code : Tests (p.33)

Test de la route Mon Compte :

Ici on crée un client qui agit comme le navigateur (`static::createClient()`) puis on va récupérer notre utilisateur en base de données. On se connecte et on vérifie s'il peut accéder à son compte (`/mon-compte`). Le terminal renvoie un statut 200 en cas de succès.

```
1 public function testMonCompteRouteWithExistingUser()
2 {
3     // Créer un client HTTP pour tester la route
4     $client = static::createClient();
5
6     // Récupérer un utilisateur existant dans la base de données (s'il existe)
7     $userRepository = self::getContainer()->get('doctrine')->getRepository('App\Entity\User');
8     $user = $userRepository->findOneBy(['email' => 'test.doe@gmail.com']); // Remplacer par l'email d'un utilisateur existant
9
10    // Vérifier si l'utilisateur existe
11    $this->assertNotNull($user, 'L\'utilisateur avec cet email n\'existe pas dans la base de données.');
```

Test de récupération des données JSON pour le chatbot :

Dans l'exemple ci-dessous, on teste que le chatbot renvoie bien les données à l'utilisateur.

```
1 class ChatbotAPITest extends WebTestCase
2 {
3     public function testGetChatbotData()
4     {
5         $client = static::createClient();
6
7         // Effectuer une requête GET vers la route
8         $crawler = $client->request('GET', '/api/chatbot-data');
```


- Annexe 29 : Snippet Terminal : test réussi / échoué (p.33)

En cas de test réussi :

```
PS C:\Users\Victor\Desktop\AAF\REPO\AlloAfroFoodV2> php bin/phpunit tests/Functional/RoutesTest.php
PHPUnit 9.6.22 by Sebastian Bergmann and contributors.

Testing App\Tests\Functional\RoutesTest
.....
8 / 8 (100%)

Time: 00:09.473, Memory: 84.00 MB

OK (8 tests, 8 assertions)
```

En cas de test échoué et/ou de composant déprécié :

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS

#11 C:\Users\Victor\Desktop\AAF\REPO\AlloAfroFoodV2\bin\phpunit(10): require('C:\\Users\\Victor...')
#12 {main}

FAILURES!
Tests: 1, Assertions: 1, Failures: 1.

Remaining self deprecation notices (2)

1x: Method "Symfony\Component\Form\AbstractType::buildForm()" might add "void" as a native return type declaration in the future. Do the same in child
or add an explicit @return annotation to suppress this message.
1x in HomePageTest::testHomePageLoadsSuccessfully from App\Tests\Functional

1x: Method "Symfony\Component\Form\AbstractType::configureOptions()" might add "void" as a native return type declaration in the future. Do the same in
errors or add an explicit @return annotation to suppress this message.
1x in HomePageTest::testHomePageLoadsSuccessfully from App\Tests\Functional
```